Chunk First: When Prefill–Decode Disaggregation Pays Off over TCP

Measurements and a Calibrated Model on Single-GPU Cloud Instances

Sunghyun Jun
Independent Researcher
[contact email]

[draft v4, 2026-10-01]

Abstract

Prefill–decode disaggregation usually relies on RDMA or NVLink, which cheap single-GPU instances lack. For a TCP-linked fleet of such instances, we ask whether disaggregation is worth considering once chunked prefill, which attacks the same decode stalls, is tuned, and if so, where. Serving Gemma 4 26B and 31B with stock vLLM and NIXL/UCX on AWS, we measured both costs: a replica's longest inter-token gap followed the chunk size (26B, 4K inputs: 48 ms at 2,560 tokens, 21 ms at 512), and disaggregation's added time to first token (TTFT) was mostly KV transfer, which a larger UCX segment made 3.4× faster at low load. At 4K inputs and in chat, against tuned chunks, disaggregation served more per GPU under none of the latency targets our measured rates could rank; it won, by about 3× for 26B, only against replicas held at 2,560 tokens, near vLLM v0.30.0's floor for Gemma 4 with video enabled. A step-time model calibrated on our runs predicts that at low load and a 50 ms gap limit, inputs of roughly 14K–37K tokens (lower edge 11K–34K within its uncertainty) need so many chunks that disaggregation over TCP meets TTFT limits of a second or more that no chunk meets, given a small KV cache. Tune the chunk first.

1 Introduction

Serving a request to a large language model takes two phases: *prefill*, which processes the whole input at once, and *decode*, which produces output tokens one at a time. When both share a GPU, a prefill scheduled into the same step as ongoing decodes lengthens that step, and every streaming request on that GPU pauses. Two remedies exist. *Chunked prefill* splits a long prefill into pieces that are batched with decodes, which bounds the work any one step carries [1, 2]. *Prefill–decode (PD) disaggregation* runs the two phases on different GPUs and ships the KV cache from the prefill GPU to the decode GPU [3–5].

Disaggregation pays for its isolation with a KV-cache transfer, and published systems run that transfer over fast links: InfiniBand [4], RDMA [5], or NVLink inside one node [3]. The cheapest single-GPU cloud instances offer none of these. On AWS, the 2xlarge sizes of the g6e, g7, and g7e families have one GPU, no Elastic Fabric Adapter (EFA), and no ENA Express [6]; the 8xlarge sizes of the same families add EFA at roughly 1.6–2× the hourly price [7, 8]. Public documentation discourages disaggregation over TCP: llm-d calls it "extremely slow" and "targeted for local development" [9], and NVIDIA Dynamo warns of a 200–500× slower TTFT without RDMA and recommends aggregated serving instead [10]. Few public measurements show how slow the TCP path is, or whether disaggregation over it still beats a well-configured colocated deployment.

Our question comes from scaling out on such instances. A fleet of single-GPU nodes linked by TCP has several levers against prefill interference and repeated prefill work: chunked prefill, disaggregation, prefix-cache-aware routing as in llm-d [11], and KV-cache offloading as in LMCache [12]. Chunked prefill and disaggregation attack the same problem, the decode stalls that other requests' prefills cause, and only disaggregation pays for its fix with a network transfer. We therefore ask whether disaggregation over TCP is worth considering at all once the chunk size is tuned, and if so, where.

We measured both remedies on the same hardware. We served two Gemma 4 models with stock vLLM on one to three single-GPU instances, disaggregated them with vLLM's NIXL connector over UCX's TCP transport, and judged each request against a service-level objective (SLO) on its TTFT and on the longest gap between its output tokens. Our first comparison used replicas at a 2,560-token chunk, the smallest our stack accepted with images disabled. Two rounds of AI-assisted mock peer review found that disabling every modality allows smaller chunks and that the replicas' request proxy capped concurrent connections at 100; we added two sessions (E8 and E9) that swept the chunk size and removed the cap, and we committed our predictions for E9 before running it. Our tuned-chunk measurements stop at 4K-token inputs, so for longer inputs we calibrate a step-time model on the measured runs and use it to locate the boundary.

We ask three research questions (RQs):

- **RQ1.** How large are the two costs on cloud hardware: the stall that a prefill chunk imposes on colocated decodes, and the TTFT that a KV transfer over TCP adds?
- **RQ2.** At the input lengths we measured (4K tokens and chat), does a tuned chunk leave any SLO in which disaggregation over TCP serves more per GPU?
- **RQ3.** Where does chunked prefill run out, and what decides whether disaggregation over TCP can take over there?

In short: a chunk of 512–1,024 tokens kept the 26B replicas' median longest gap at 21–28 ms, and disaggregation's added TTFT was 61–85% KV transfer at low load (RQ1). At 4K inputs and in chat, no SLO that our measured rates could rank favored disaggregation; it won only when the chunk could not shrink (RQ2). Our model predicts that at low load chunking runs out for long inputs, from roughly 14K tokens at a 50 ms gap limit for 26B FP8 (11K–34K within its uncertainty), where disaggregation over TCP can meet TTFT limits of a second or more that no chunk meets, given a small KV cache (RQ3).

Our contributions are:

- measured sizes of both costs on rented single-GPU nodes, including the TCP transfer path: the UCX segment size, NIXL's splitting of a transfer over up to four connections, the cloud's per-flow cap, and a KV-volume and TTFT model (Section 4);
- a head-to-head at equal GPU count with tuned chunks, a map over 192 SLOs, and the exception when the chunk cannot shrink (Section 5);
- a step-time model, calibrated on measured runs and checked against measurements it was not fitted to, that estimates, with its uncertainty, where chunked prefill runs out and shows how input length, gap limit, KV size, and link speed move the crossover (Section 6);
- a procedure that places disaggregation among chunk tuning, prefix-aware routing, and KV offloading (Section 7.1), the measurement pitfalls we met (Appendix B), and the harness, per-request data, and four rounds of mock review.

2 Background and Related Work

Disaggregated serving. DistServe splits prefill and decode, provisions each phase for its own latency target, and measures per-GPU goodput at an SLO attainment target [3]; we use the same metric. With 25 Gbps between nodes, DistServe keeps the KV transfer on intra-node NVLink. Splitwise runs the phases on different machine types over InfiniBand and overlaps the transfer with prefill layer by layer [4]. Mooncake builds a KV-cache-centric architecture over RDMA [5]. TetriInfer [13], DéjàVu [14], P/D-Serve [15], and KVDirect, which pulls the KV cache from the decode side as vLLM's connector does [16], belong to the same family and assume a fast interconnect.

Colocation and when to split. SARATHI introduced chunked prefill [1], and Sarathi-Serve built stall-free batching on it, trading prefill throughput against decode stalls through the chunk size [2]. semi-PD disaggregates compute inside one GPU while sharing KV storage [17]. TaiChi unifies the two with prefill-heavy and decode-heavy instances whose ratio and chunk sizes are set by configurable sliders, and reports that aggregation suits tight TTFT and relaxed time per output token (TPOT) while disaggregation suits strict TPOT and relaxed TTFT [18]. vLLM's documentation states that disaggregated prefill does not improve throughput, motivates it by tail inter-token latency, and notes that chunked prefill with a proper chunk size can reach the same goal but that the right size is hard to find [19]. We test that trade-off with stock vLLM over cloud TCP. Our conclusion leans against disaggregation more than TaiChi's; candidate reasons are the metric (each request's longest gap rather than its mean per-token time), the interconnect, and the models and scale, which we did not separate. EcoServe targets clusters with commodity interconnects, where full disaggregation depends on fast links, and proposes partial disaggregation [20]; ThunderServe and HexGen-2 schedule phase-split serving across heterogeneous cloud GPUs and networks [21, 22].

KV transfer layer. vLLM [23] moves the KV cache with NIXL [24], which wraps transports including UCX [25]; without RDMA, UCX uses TCP. CacheGen compresses the KV cache to reduce network traffic [26]. Prefill-as-a-Service sends the small KV caches of hybrid-attention models between datacenters over commodity Ethernet [27], and PDD disaggregates across clusters linked by wide-area TCP [28]. The llm-d project measured GPU-to-GPU transfers over 100 Gbps TCP at 0.55 GB/s with UCX and 4.7–4.9 GB/s with UCCL [29]. Mitra et al. find disaggregation most effective for prefill-heavy traffic and larger models [30].

Reusing and placing KV caches. Prefix-cache-aware routing sends a request to the replica that already holds its prefix, so the prefill shrinks without moving any KV cache; llm-d's scheduler scores replicas by KV-cache affinity [11]. LMCache stores KV caches outside the engine and shares them across engines and queries [12]; loading a KV cache from another node is a transfer of at least the same size as a disaggregated one. We did not measure either; Section 7.1 uses our transfer measurements to reason about them.

Metrics. Etalon argues that TTFT, time between tokens, and their percentiles miss what users experience and proposes a fluidity index [31]; Andes defines a quality of experience over each request's token delivery timeline [32]. We use a simpler per-request criterion, the longest gap between consecutive output tokens.

Where this work sits. We do not design a system. We measure stock vLLM on rented single-GPU nodes, compare disaggregation with colocated replicas whose chunk size we tune, explain the

transfer path, and calibrate a simple model to say where the comparison turns. We did not find a measurement of this setting, but our search was not exhaustive.

3 Methodology

3.1 Environment

We used three single-GPU instance types in one availability zone of AWS us-east-1 (Table 1). None of these sizes supports EFA or ENA Express [6], so all traffic between nodes is TCP. Within the zone, one TCP flow reached at most 4.97 Gbps by iperf, matching the documented 5 Gbps single-flow limit [33]. Inside a cluster placement group, an AWS feature that places instances close together, one flow reached 9.54 Gbps (documented: 10 Gbps), and eight parallel flows reached 49.72 Gbps, the g7e's nominal 50 Gbps. Load generation and request routing ran on a separate CPU instance (c6i.xlarge before session E7, c6i.2xlarge from E7 on).

Instance	GPU	GPU memory	$/hour	Used for
g7.2xlarge	RTX PRO 4500	32 GB	–	input-length comparison, first transfer tuning, bandwidth limit, cross-GPU comparison
g6e.2xlarge	L40S	48 GB	2.242	cross-GPU comparison
g7e.2xlarge	RTX PRO 6000	96 GB	3.363	placement group, chunk sweeps, head-to-head, scaling, repeats

Table 1: GPU instances (one GPU each, no EFA). Prices are on-demand in us-east-1; costs in this paper use the g7e price. We did not record the g7 price, and no cost in this paper depends on it.

We used vLLM v0.30.0 (NIXL 1.4.1) with NIXL's pull-based connector, in which the decode node reads the KV cache from the prefill node. Unless stated otherwise, disaggregated runs used seven UCX environment variables that we tuned in early sessions (Appendix C), except the three E9 runs with an 8,192-token prefill chunk, which set only the two segment sizes; Servers used a GPU memory utilization of 0.90 (except 0.42 in the KV-matched single-node runs and 0.70 in the host-staging runs), at most 256 concurrent sequences, and a 32,768-token context (16,384 for the 31B FP8 cross-GPU runs of E1). The disaggregated side kept its defaults: a 2,560-token chunk on both nodes (the three runs above used 8,192 on the prefill node), 256 sequences on the decode node, and NIXL's default four worker threads.

3.2 Models and KV size

We served two Gemma 4 models [34]. 26B-A4B is a mixture-of-experts model that activates 3.8B of its 25.2B parameters per token; 31B is dense (30.7B parameters). Both interleave sliding-window and global attention. We mainly used NVIDIA's NVFP4 checkpoints, whose KV cache is FP8 [35], and for comparison FP8 (weights and activations, with dynamic activation scales) and BF16 checkpoints of 26B, whose KV cache is BF16. Because sliding-window layers keep only the most recent tokens, the transferred KV volume has a global part that grows with the input length L and a local part that stops growing at the window W:

$$V(L) = b_{\mathrm{g}}\, L + b_{\mathrm{w}} \min(L, W). \tag{1}$$

For 26B NVFP4, vLLM reported 140, 260, and 412.5 MiB per request at 4K, 16K, and 32K inputs, which gives $b_{\mathrm{g}} \approx 10$ KiB per token and a saturated local part of about 100 MiB, consistent with

$W = 1{,}024$ and $b_w \approx 100$ KiB per token; chat requests (median input about 100 tokens, mean about 230) moved 25–26 MiB on average, as Eq. (1) predicts from their input lengths. The FP8 and BF16 checkpoints transfer twice as much (280 MiB at 4K); 31B NVFP4 transfers 560 MiB at 4K. vLLM reports these volumes as "MB"; they are mebibytes, and we convert rates as Gbps = $0.008389 \times$ MiB/s.

3.3 Configurations

We compared two approaches at equal GPU count. In the *replica* configuration, each node runs a colocated server that does both prefill and decode, and a round-robin proxy spreads requests over the nodes. Replicas use chunked prefill; the *chunk* is vLLM's maximum number of tokens per scheduler step. In the *disaggregated* configuration there are prefill-only (P) and decode-only (D) nodes, written xPyD for x prefill and y decode nodes.

Three practical details shaped the comparison. First, for Gemma 4, vLLM rejected chunks smaller than 2,496 tokens, the budget of one video item, with images disabled or with images and audio disabled, and accepted them with images, audio, and video all disabled (every modality set to 0 in `--limit-mm-per-prompt`); we did not test other combinations. Sessions before E8 therefore used 2,560 tokens; E8 swept 256–2,048 and E9 added 512 for 26B. With every modality disabled, a 2,560-token chunk failed at engine start, so the 2,560 controls kept images disabled only. Second, until E9 the replicas' proxy, a small aiohttp program, allowed at most 100 concurrent connections, and requests beyond that waited in the proxy; the eleven replica runs that reached the cap are caution data (Appendix B.1). Third, disaggregated runs before E8 used vLLM's test proxy for NIXL (FastAPI with httpx); E8 and E9 added an aiohttp proxy with the same request flow and no connection limit (Appendix B.1).

3.4 Workloads

Chat uses ShareGPT conversations [36] (median input 76–141 and median output 24–177 tokens, depending on the sampled subset); prompts went to the completions endpoint without a chat template, and with end-of-sequence ignored each output had the length of the reference answer. *Long input* uses synthetic requests with 4,096, 16,384, or 32,000 input tokens and 512 output tokens. *Trace* replays 600 requests of the conversation trace released with Splitwise (median input 996, output 217 tokens) [37]. Requests arrive as a Poisson process. A measurement point at a total rate of r requests/s sends $\max(100, \lceil 180r \rceil)$ requests, about 180 s at 0.56 requests/s or more. The same rate and repeat index use the same seed in every configuration, so comparisons at a rate are paired. We report rates per GPU unless stated otherwise.

3.5 Metrics and SLO

We judge each request individually. Under the *strict SLO*, a request must have TTFT (time from sending the request to the first output token) of at most 1 s and a longest inter-token gap of at most 50 ms; the *relaxed SLO* uses 3 s and 250 ms. We use the longest gap because one long pause in a 500-token answer is 0.2% of its gaps and vanishes from percentile statistics, yet it is the pause a user sees. A request with a single output token has no gap and is judged on TTFT only, a rule we adopted after session E8; it turned two borderline 31B 1P1D runs at 4 requests/s per GPU from failing (0.899 and 0.878) to passing (0.924 and 0.903) and raised other chat runs by at most 0.055.

Attainment is the fraction of requests that meet the SLO. *Supported load* is the highest measured rate at which every repeat reaches 0.9 attainment. *SLO goodput* is the output-token throughput of

SLO-meeting requests (the minimum over repeats at that rate), and *cost* is the GPU hourly price divided by SLO goodput, in dollars per million tokens.

The thresholds are design choices. TTFT targets in prior work range from 0.125 s to 30 s [3, 5]; 1 s is the classic limit for keeping a user's flow of thought [38]. For the gap, our pre-registered protocol (Appendix D) used five times each model's decode step, Sarathi-Serve's strict rule [2]. We switched to a common absolute limit because a relative rule makes faster models stricter in absolute terms (38 ms for 26B BF16 on g7e) and makes models hard to compare; 50 ms is five times the 26B decode step on the slowest GPU we used. Section 5.2 shows how each verdict changes over a grid of TTFT and gap limits.

3.6 Procedure, data selection, and statistics

All reported runs had the prefix cache disabled. Some early runs had it enabled while replaying the same seeded requests, so later points hit cache entries left by earlier ones; we excluded those 122 runs, 5 server-start failures, and 1 discarded run, 128 of 455 runs. Fourteen runs are caution data and not used for verdicts: 5 in which the client-side proxy reached its CPU limit (two of them replica runs that also reached the connection cap) and 9 replica runs that reached the proxy's connection cap. The remaining 313 runs are the *audited* runs. Sessions E7–E9 warmed every server with 30 requests; earlier sessions did not, and dropping each run's first five requests raised attainment by at most 0.019, lowered it by at most 0.029, and flipped no verdict.

Median TTFT varied by up to 10% across repeats. Points well above 0.9 attainment varied by at most 0.03 within a session; points near 0.9 varied by up to ± 0.10 and could flip a verdict, and we call a point *borderline* if any repeat falls between 0.85 and 0.95; low points varied more (0.37–0.64). For decisive points we report 95% intervals from a hierarchical bootstrap that resamples repeats and then 20 time blocks within each run (Appendix F). Disaggregated runs with vLLM's test proxy, whose garbage collection paused streams (Appendix B.1), remain in the data; where both proxies were measured, verdicts use the better of the two. Disaggregated serving gave the same outputs as a single node, within the nondeterminism that 26B showed even on one node (Appendix A). We wrote the predictions for session E9 into our lab notebook and committed them before the session (Appendix E). Appendix G maps sessions to data folders. Section 6 fits a model to existing runs only and adds no measurements.

4 Measured costs

4.1 System percentiles hide per-request stalls

The system-wide p99 of inter-token latency pools every gap of every request, and it can look healthy while many requests still stall (Figure 1). Of the 215 runs with TTFT p99 below 1 s, 28 had system p99 at or below 50 ms but attainment below 0.9. For example, two g7e replicas serving 26B FP8 with 4K inputs at 0.25 requests/s per GPU had a system p99 of 8.6 ms, yet 48% of requests had at least one gap over 50 ms. All later verdicts are per request.

4.2 The chunk size sets the replicas' stall

A replica stalls when another request's prefill joins the step, and the stall lasts as long as that step. With 4K inputs and 512 outputs almost every request overlaps another request's prefill, so its longest gap is roughly the time of a step that carries a full chunk. For 26B NVFP4 that time fell with the chunk (Figure 2a): the median longest gap was 48.4 ms at 2,560 tokens, 38.8 ms at

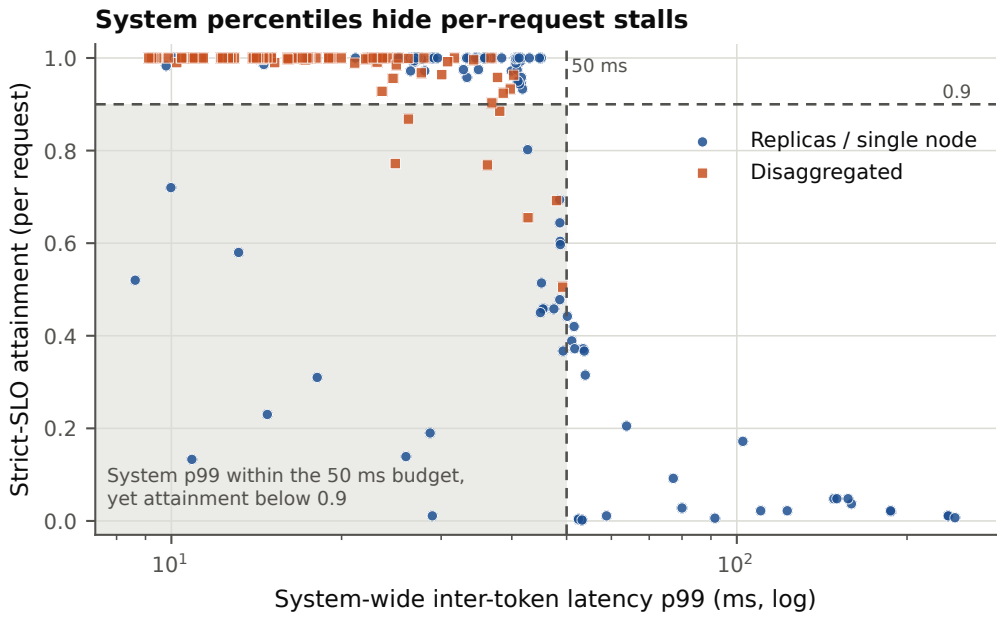


Figure 1: System-wide inter-token latency p99 (log scale) against per-request strict-SLO attainment for the 215 runs (212 audited, 3 caution) with TTFT p99 below 1 s. The shaded region holds runs whose system p99 is within 50 ms yet whose attainment is below 0.9: 21 replica or single-node runs and 7 disaggregated runs (2 of them caution).

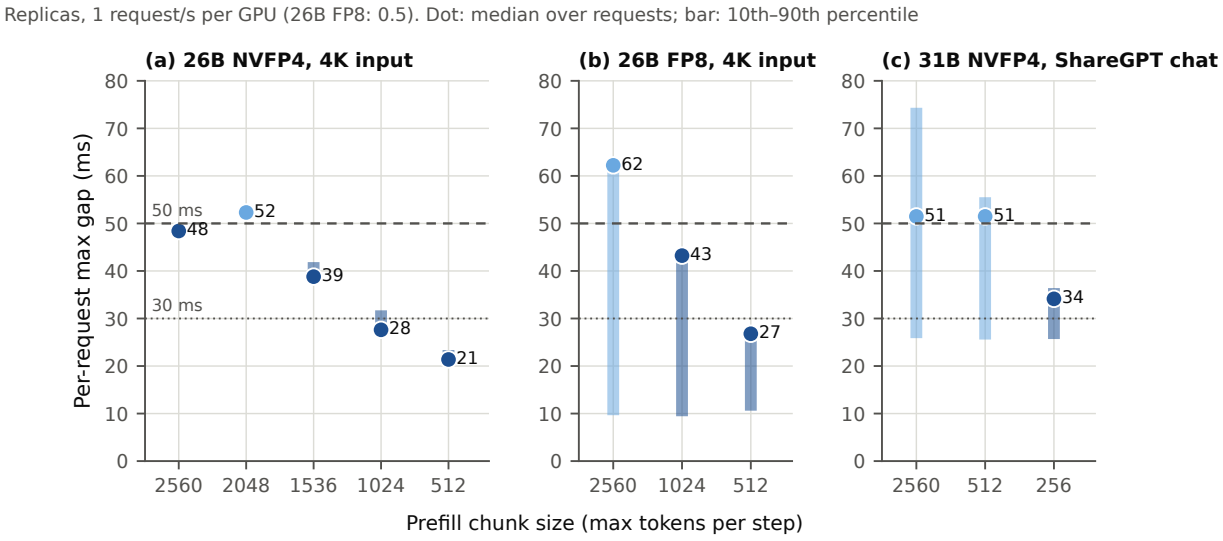


Figure 2: Per-request longest gap of colocated replicas by chunk size (two g7e nodes in a placement group). Dot: median over requests; bar: 10th–90th percentile; light dots have a median above 50 ms; dashed and dotted lines mark 50 and 30 ms. (a) 26B NVFP4, 4K input, 1 request/s per GPU. (b) 26B FP8, 4K input, 0.5 requests/s per GPU. (c) 31B NVFP4, ShareGPT chat, 1 request/s per GPU. The 2,560-token points pool the repeats of sessions E5, E7, and E8 in (a) and (c); in (b) it is one E7 run. The 512-token points in (a) and (b) come from E9.

1,536, 27.6 ms at 1,024, and 21.4 ms at 512, while median TTFT rose from 132 to 205 ms. Between 1,024 and 2,048 tokens the step grew by about 24 µs per token; below 1,024 it shrank more slowly toward the decode-only step, about 9 ms at this load. The 2,560-token median sat 1.6 ms under the 50 ms limit, so the replicas' verdicts at that chunk hinged on a small margin. A 2,048-token chunk gave 52.3 ms, longer than 2,560; we could not tell whether the even split of a 4K input into two 2,048-token pieces or the different multimodal setting of the 2,560 control causes this.

Heavier FP8 weights make each step longer: at 0.5 requests/s per GPU the median longest gap was 62, 43, and 27 ms at 2,560, 1,024, and 512 tokens, with attainment 0.13, 0.99, and 1.00 (Figure 2b). For 31B chat the longest gap depends on the longest prompt prefilled on the same replica while a request decodes (Figure 2c). In the five 2,560-token runs at 1 request/s per GPU, 99–100% of requests kept every gap under 50 ms when that prompt had fewer than 200 tokens, 42% with 200–400 tokens, 12% with 400–800, and almost none above 800 (1 of 323). A 512-token chunk therefore did not help (attainment 0.44, the same as 2,560), and a 256-token chunk did (1.00 in two runs, median longest gap 34 ms, TTFT unchanged at 79 ms).

4.3 The KV transfer over TCP

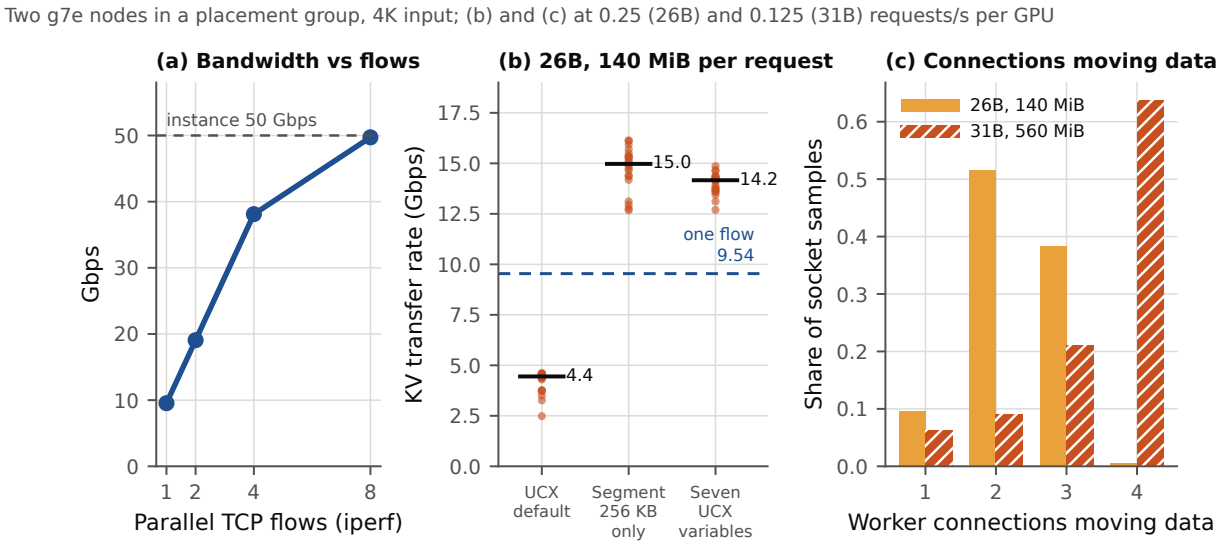


Figure 3: (a) Bandwidth between two g7e nodes in a placement group against the number of parallel TCP flows (iperf). (b) KV transfer rate of 26B NVFP4 (140 MiB per 4K request, 0.25 requests/s per GPU) under three UCX settings; dots are 10 s windows reported by vLLM, black lines are medians, and the dashed line is one flow's limit. The seven-variable run is the one with our aiohttp proxy. (c) Share of socket samples (about 0.12 s apart, longer than a 26B transfer of about 80–90 ms) in which 1–4 of the four worker connections moved more than 64 KiB, among samples in which any moved; the shared fifth connection is not counted (26B at 0.25 and 31B at 0.125 requests/s per GPU).

Three things set the transfer rate: a software default, how NIXL splits a transfer, and the cloud's caps. With UCX defaults, the 8 KB TCP segment limited the transfer to a median of 4.45 Gbps, less than half of one flow's limit (Figure 3b). Raising only the send and receive segment sizes to 256 KB lifted it to 14.97 Gbps, a 3.4× gain at low load. The full set of seven UCX variables gave 13.5–14.2 Gbps, 5–10% lower: it requested 8 MB socket buffers, and in both seven-variable runs we sampled, the receiver's TCP window scale was 2 (7 with the segment sizes alone or the defaults), which caps the advertised window at 256 KiB; we infer that the kernel capped the requested buffers

when the connection opened, but we did not record its limits. The sender was limited by the receive window for about 40% of its busy time. In our first tuning session, on g7 nodes without a placement group, the seven-variable change cut median TTFT at 0.125 requests/s per GPU from 626 to 466 ms and raised attainment at 0.5 from 0.82 to 0.99.

NIXL (version 1.4.1) hands a transfer to up to four worker threads, each with its own UCX endpoint and TCP connection: a 26B 4K transfer goes out in two or three pieces, a 31B 4K transfer in four, and a short one whole on a shared connection (Appendix C). Socket samples agree (Figure 3c): four connections each carried 22–27% of the bytes and a fifth, shared one carried the rest; in most samples during 26B transfers two or three connections moved data, and in 64% of the samples during 31B transfers all four did.

We express the rate relative to the per-flow cap as $m = \text{effective rate}/C_{\text{flow}}$, taking the median over vLLM's 10 s windows (at the loads that decided verdicts, windows with at least five transfers). At low load m was 1.4–1.6 for 26B and 1.8 for 31B. At the loads where our verdicts were decided it fell to 0.92–1.14 for 26B (3–4 requests/s per GPU; 0.68–0.81 in the 8,192-token prefill runs). About a third of 31B chat transfers (three quarters of the bytes) were split, and they ran at about 0.8 (0.79–0.82 at 4 requests/s per GPU, 0.57 at 6). Under load, the time vLLM reports includes waiting for other transfers, so m there describes the queue rather than the link. With the seven-variable set, under which the sender was receive-window-limited even with four connections active, 31B transfers aggregated 19–21 Gbps, about half of what four iperf flows reached (38 Gbps), which points to a shared limit that we did not identify. More connections alone are therefore not a sure lever.

Other transfer settings did not help: staging the KV cache through host memory, push transfer, and a proposed patch that overlaps the transfer with prefill (Appendix C).

4.4 The first-token cost of disaggregation

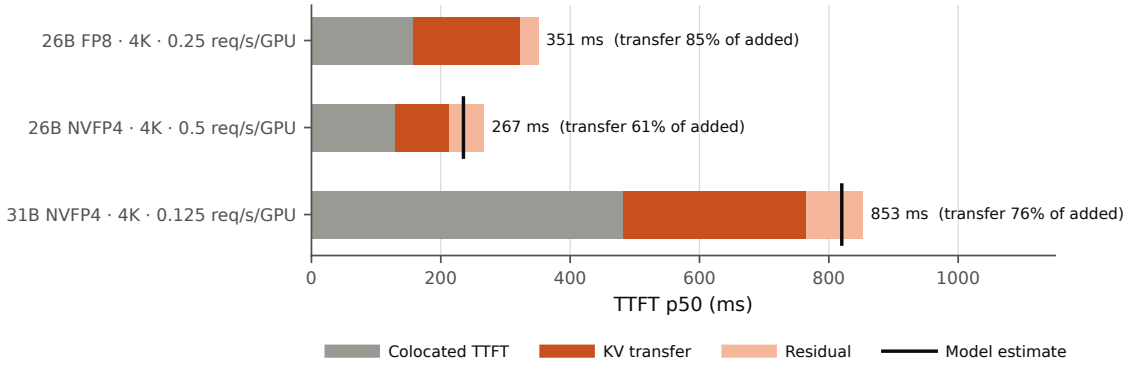


Figure 4: Median TTFT of 1P1D split into colocated TTFT at the same load, mean KV transfer time (the mean of vLLM's 10 s window averages), and residual (placement group). Black ticks are estimates from Eq. (2). The first row joins measurements from two days.

Disaggregation adds TTFT, and most of it is the KV transfer (Figure 4): 61% of the added time for 26B NVFP4 at 0.5 requests/s per GPU (84 of 137 ms), 76% for 31B at 0.125 (283 of 370 ms), and 85% for 26B FP8 at 0.25. The residual, 28–87 ms, covers the decode node's first step, scheduling, and the proxy hop, and it would remain with RDMA. At low load the relation is

$$\text{TTFT}_{\text{PD}} \approx \text{TTFT}_{\text{coloc}} + \frac{V(L)}{m \times C_{\text{flow}}} + \text{residual}, \tag{2}$$

with $V(L)$ from Eq. (1), C_{flow} the per-flow cap, and m as defined above. We wrote estimates from this relation into our lab notebook before the three-node session: 235 ms for 26B at 0.5 requests/s per GPU (measured 267 ms, 14% higher) and 810–830 ms for 31B at 0.125 (measured 853 ms); unlike the E9 predictions, these were not committed separately, so the repository does not record their order. The relation omits queueing: when we limited the sender's aggregate bandwidth to 1 Gbps (Appendix Figure A1), it predicted about 1.5 s against a measured median of 2.19 s; an M/D/1 waiting time (transfer 1.17 s, utilization 0.59) adds 0.83 s on average, giving about 2.3 s. Eq. (2) is a low-load estimate, and m falls with load. It also shows why long inputs fail: for 26B FP8 at 16K inputs, about 520 MiB take roughly 0.3–0.7 s on top of a prefill of about 0.8 s, and we measured 1P1D's TTFT 465–506 ms above the replicas', beyond a 1 s TTFT limit.

Answer to RQ1. Both costs are of the expected kind; what we add is their size. The chunk sets the stall: for 26B with 4K inputs the median longest gap fell from 48 to 21 ms as the chunk shrank from 2,560 to 512 tokens, at a median TTFT cost of about 30 ms at 1,024 tokens and 70 ms at 512 (1 request/s per GPU); for dense 31B chat the chunk had to be 256 tokens. Disaggregation's added TTFT was 61–85% KV transfer at low load. A 256 KB UCX segment made the transfer 3.4× faster, after which the per-flow cap and NIXL's splitting set its rate, and under load transfers queued.

5 Head to head at measured input lengths

5.1 Tuned chunks against disaggregation

Workload	Replicas, chunk 2,560	Replicas, best chunk	1P1D (aiohttp proxy)
26B NVFP4, 4K	1†, \$2.00, 132 ms	5 (1,024), \$0.38, 259–278 ms	3, \$0.63, 366 ms (failed 4 narrowly)
26B FP8, 4K	none	≥1 (512 or 1,024), ≤\$1.87, 199–247 ms	≥1, ≤\$1.88, 384 ms
31B NVFP4, chat	none	≥8 (256), ≤\$0.67, 117 ms	4†, \$1.67, 303–308 ms

Table 2: Supported load under the strict SLO (requests/s per GPU; every repeat ≥0.9), cost per million SLO-meeting output tokens at that load (g7e, \$3.363 per GPU-hour, client excluded), and median TTFT at that load. "≥" and "≤": the configuration still passed at the highest rate we measured; for 31B replicas that is one run at 8, and the run at 6 reached the proxy's connection cap and is caution data. † Passed, but borderline (a repeat within 0.85–0.95). The 26B FP8 1P1D value uses vLLM's test proxy, the only one measured; with the test proxy, 31B 1P1D failed at 4 (0.885).

Against replicas at a tuned chunk, disaggregation did not serve more. For 26B with 4K inputs, 1,024-token replicas supported 5 requests/s per GPU and 1P1D 3. Without the proxy cap, the replicas passed 4 and 5 in every repeat and collapsed at 6 (0.19), with a small margin at 5 (longest gaps of 46.3 and 47.1 ms); 512-token replicas passed 1–4. 1P1D was borderline at 4 (0.885–0.992 with our aiohttp proxy, 0.77–0.98 with the test proxy) and collapsed at 5 (0.18) as prefills queued and gaps grew. Appendix Table F1 lists every repeat with its interval.

A larger prefill chunk did not change these verdicts. Raising it to 8,192 tokens, in runs that also set only the UCX segment sizes, cut 1P1D's median TTFT at 4 requests/s per GPU from about 500 to 416–451 ms, but attainment stayed at 0.874 and 0.976 at 4 and 0.365 at 5. Its transfers were slower than with the seven-variable set at the same seeds (transfer-weighted means of 165–190 against 119–140 ms), so the gain came from the prefill side. At 5 the prefill node kept up, and KV

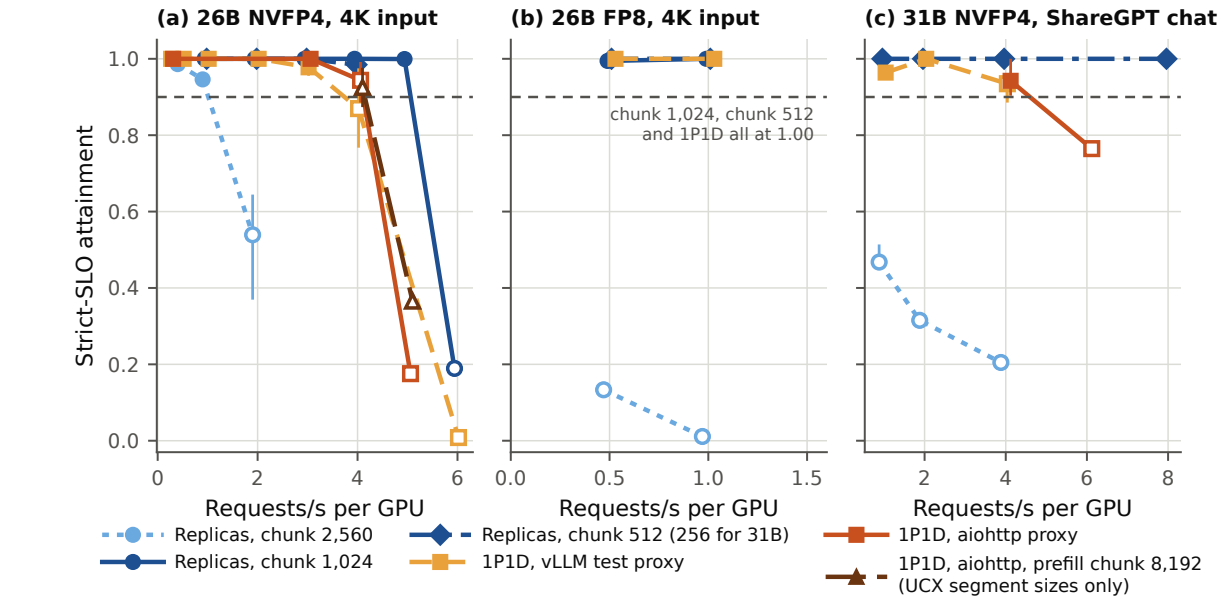


Figure 5: Strict-SLO attainment against load on two g7e nodes in a placement group. Blue: replicas at a 2,560-token chunk (light, dotted), 1,024 (dark, solid), or 512 for 26B and 256 for 31B (dark, dash-dot); runs that reached the proxy's connection cap are excluded, and the 1,024-token points at 4 requests/s per GPU and above come from E9. Orange and brown: 1P1D with vLLM's test proxy (light, dashed), our aiohttp proxy (dark, solid), or our aiohttp proxy with an 8,192-token prefill chunk (brown, long dashes; these runs set only the UCX segment sizes). Lines connect the mean of repeats; vertical bars span the minimum and maximum; series are shifted slightly along the x axis so that the bars do not overlap. Hollow markers: some repeat below 0.9, so the point fails the verdict rule. The test-proxy line in (a) joins 4 and 6 requests/s per GPU directly (5 was not measured). The dashed horizontal line is 0.9. The 2,048- and 1,536-token replicas are omitted for readability.

transfers queued instead (transfer-weighted mean 0.92 s against 0.18 s with a 2,560-token prefill chunk) at an offered KV traffic of 11.7 Gbps. Because these runs changed two settings, we do not attribute the slowdown.

The decode node's double load did not lengthen the gaps that decided the 26B verdicts; for 31B chat it slowed the decode steps. At equal GPU count, 1P1D's decode node carries the decodes of both GPUs. For 26B with 4K inputs, at 3 and 4 requests/s per GPU its mean time per output token (15.7 and 20.5–20.9 ms) matched that of 1,024-token replicas (16.7 and 20.7–20.9 ms), and its median longest gap was shorter (27.6 against 33.8 ms at 3), because no prefill entered its decode steps; it failed at 4 on the TTFT tail (p99 0.97–1.25 s against 0.39–0.46 s) and on rare decode-server pauses (Appendix B.1). For 31B chat the fixed 1:1 split was unbalanced: short prompts kept the prefill GPU's streaming multiprocessors active only about 30% of the time at 4 requests/s per GPU (41% at 6) against 100% on the decode node, and at 4 the decode node's mean time per output token was 32–33 ms against 30 ms for 256-token replicas.

For 31B chat, tuned replicas supported at least twice the load of 1P1D. 256-token replicas passed every measured load up to 8 requests/s per GPU (1.000 at 8, longest gap 48.1 ms), while 1P1D passed 4 narrowly with our aiohttp proxy but not with the test proxy, and failed at 6 (0.765). For 26B FP8, 512- and 1,024-token replicas and 1P1D all passed the highest load we measured, 1 request/s per GPU. At every load where both passed, tuned replicas had a lower median TTFT (78–247 ms against 255–384 ms for 1P1D); at 4 requests/s per GPU for 26B, where 1P1D was borderline, 200–359 against 416–526 ms. Two proxy artifacts distorted earlier comparisons in both directions: vLLM's test proxy for NIXL paused all streams during its garbage collection, and the replicas' proxy capped connections at 100, which lowered their supported load by one step; Appendix B.1 details both.

5.2 No SLO we could rank favors disaggregation against a tuned chunk

Because every per-request timing is stored, we re-judged all runs over a grid of TTFT limits (0.25–5 s) and longest-gap limits (30–300 ms) and compared the best configuration of each kind (Figure 6). The top row, against 2,560-token replicas, is discussed in Section 5.3. Against the best measured chunk (bottom row), no cell favors disaggregation:

- For 26B NVFP4, replicas serve more in 55 of 64 cells, including 30 ms, where 512-token replicas passed 3 requests/s per GPU and 1P1D passed 2 (borderline). In eight cells (TTFT 3–5 s, gaps of 100 ms or more) both sides passed 5 and failed 6 requests/s per GPU (1P1D's prefill queue diverged at 6, median TTFT 14.6 s), so we count them as ties; one cell cannot be ranked.
- For 26B FP8, replicas serve more when the TTFT limit is 0.5 s or less; elsewhere both sides passed the highest load we measured (1 request/s per GPU), so the map cannot rank them. At 30 ms, 512-token replicas reached 0.958 at that load.
- For 31B chat, replicas serve more at 40–50 ms; at 30 ms neither side met the SLO, and at 75 ms and above most cells with a TTFT limit of 0.75 s or more cannot be ranked.
- Replicas are the only option when TTFT must stay under 0.25 s, because disaggregation's TTFT includes an 80–300 ms transfer (Section 4.4).

Of the 192 cells in the bottom row, replicas served more or alone met the SLO in 99, neither side met it in 8, the sides tied in 8, and none favored disaggregation. The remaining 77 cells, 48 of them 26B FP8, which we measured only up to 1 request/s per GPU, could not be ranked. In 12 of the 26B cells where replicas served more, the goodput ratio was 0.796–0.797 (5 against 4 requests/s per

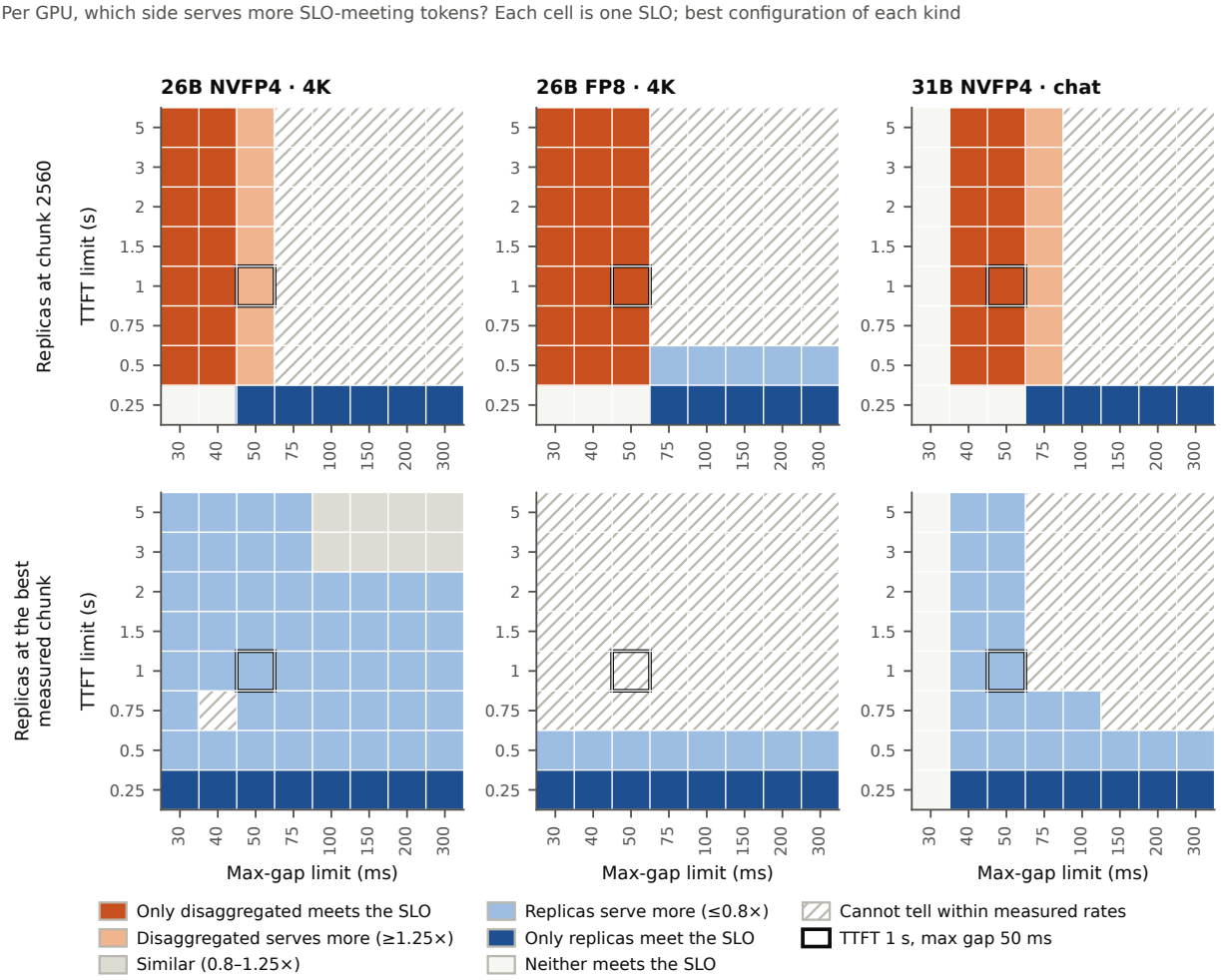


Figure 6: Which approach serves more SLO-meeting tokens per GPU as the SLO changes. Each cell is one SLO (vertical: TTFT limit; horizontal: longest-gap limit), judged at the highest rate where every repeat reaches 0.9 and with the best configuration of each kind; ratios are disaggregated ÷ replica SLO goodput. Top row: replicas restricted to a 2,560-token chunk. Bottom row: replicas at any chunk we measured (512–2,560 for 26B, 256–2,560 for 31B). Hatched: at least one side still passed at its highest measured rate, so the ratio could not be placed in one category; to decide this, runs of one serving configuration with different proxies or sessions are pooled. The outlined cell is the strict SLO.

GPU), just below the 0.8 edge of the equivalence band. With a mean-over-repeats rule, or with the equivalence band widened to one rate step (0.75–1.33), no cell favors disaggregation.

The map picks the best configuration of each kind and cannot decide cells where a side still passed our highest rate. It covers 4K and chat inputs; at 16K and longer inputs, replicas with a small chunk would need many steps per prefill, and we did not measure that regime with a tuned chunk.

5.3 When the chunk cannot shrink

The comparison turns when the chunk is held at 2,560 tokens. In vLLM v0.30.0, Gemma 4 rejected chunks below 2,496 tokens, one video item's budget, in both configurations we tested that kept video input enabled (Section 3.3), so a service that accepts video is likely in this position; we did not test settings that lower that budget, or whether image or audio input alone forces it. Measured against replicas at a 2,560-token chunk, disaggregation looked decisive (Figure 5, dotted lines). For 26B with 4K inputs, replicas passed 1 request/s per GPU narrowly (0.933–0.958 in five runs) and failed at 2 (0.37–0.64), while 1P1D passed 3; the cost per SLO-meeting token was \$2.00 against \$0.63 (Table 2). For 26B FP8 and for 31B chat, 2,560-token replicas met the strict SLO at no measured load, while 1P1D did. In the top row of Figure 6, disaggregation wins at gap limits of 30–50 ms for both 26B variants and 40–75 ms for 31B chat whenever TTFT may exceed 0.25 s. We measured this case with images disabled only, not with multimodal traffic itself.

Results from earlier sessions that used 2,560-token replicas (three nodes, input length, weight precision, a real trace, other GPUs, and output correctness) are in Appendix A.

Answer to RQ2. No. At 4K inputs and in chat, replicas with a 256–1,024-token chunk left no SLO that our measured rates could rank in which disaggregation over TCP served more, down to 30 ms gaps, and its median TTFT was always higher. Disaggregation won only against replicas held at 2,560 tokens, about 3× for 26B with 4K inputs, which is the position of Gemma 4 with video input enabled in vLLM v0.30.0.

6 Where chunked prefill runs out

Our tuned-chunk measurements stop at 4K inputs. To see what happens beyond, we fit a model of the scheduler step to the measured runs, check it against measurements it was not fitted to, and use it to map which side can meet both limits. Unlike Section 5, the map asks which side can meet both limits at low load, not how much load each serves per GPU. It is a prediction, not a measurement, and this analysis was not part of our pre-registered protocol.

6.1 A step-time model

A replica processes a prompt of L tokens in chunks of c tokens. A step that carries a chunk starting at prompt position p, together with the ongoing decodes, takes

$$t(c, p) = d + e + a\,c + b\,c\,(p + c/2), \tag{3}$$

where d is the decode share of the step, e the fixed cost of a step that carries prefill, a the per-token cost, and b the cost of attending to the prefix and within the chunk. The longest gap of a request decoding alongside is the longest such step, which for long prompts is the last full chunk. At low

load, without queueing, the replica's TTFT is the sum of its steps plus a fixed overhead o:

$$\mathrm{TTFT}_{\mathrm{rep}}(L, c) \approx o + \sum_k t(c_k, p_k) = o + n\,(d + e) + a\,L + b\,L^2/2, \tag{4}$$

with $n = \lceil L/c \rceil$ steps. The chunk enters the TTFT only through the step count, but it enters the longest gap directly, and the longest gap grows with L because later chunks attend to a longer prefix. A chunk small enough for the gap limit at long L therefore needs many steps, and each step pays $d + e$. For 1P1D we use Eq. (2) with the prefill node running 8,192-token chunks: the same $aL + bL^2/2$ plus the transfer $V(L)/(m \times C_{\mathrm{flow}})$ and a 28 ms residual. We use $m = 1.5$, the low-load rate we measured for this checkpoint and the value checked in Table 3; Figure 7 also shows $m = 1$, the rate at the loads that decided our verdicts, and $m = 1.8$, the low-load rate of transfers split four ways, as 26B transfers are above about 16K tokens. On the 1P1D side the gap limit applies to the decode node, whose longest gap grew with input length while its median inter-token latency did not (p90 14.4 ms at 4K and 27.8 ms at 16K; median 7.8 and 8.0 ms). We model it as the line through these two points; on g7 the same construction for 26B NVFP4 predicted the 32K value (43.3 ms, measured 43.3 ms). We did not identify the cause.

We fitted e, a, and b by least squares to the longest gaps of 26B FP8 on g7e (the 90th percentile of the per-request longest gap, which at these rates is a step carrying a full chunk): chunks of 2,560, 1,024, and 512 tokens at 4K inputs, and 2,560 tokens at 16K inputs. We set d to the measured median inter-token latency of those runs (7.0 ms) and o from one TTFT (4K, 2,560 tokens). The fit gives $e = 9.0$ ms, $a = 15.1$ µs per token, and $b = 2.65$ ns per token^2. We use 26B FP8 because it is the only model for which we measured 16K inputs on g7e; its BF16 KV cache is twice that of NVFP4 ($b_{\mathrm{g}} = 20$ and $b_{\mathrm{w}} = 200$ KiB per token in Eq. (1)). Table 3 compares the model with six measurements on g7e that it was not fitted to (replica TTFTs at three points within 8%, 1P1D TTFTs at three points within 7%) and with four on g7.

Beyond 16K we have only 26B NVFP4 runs on the slower g7 GPU at a 2,560-token chunk (session B). A two-parameter version of the model, solved on the 4K and 16K gaps, predicted the 16K and 32K TTFTs within 8% and 1% and overestimated the 32K gap by 7% (Table 3). This tests the attention term to 32K, not the per-step cost that decides the crossover. At 16K the model underestimated the replicas' TTFT on both GPUs (by 7–8%); we do not know why.

6.2 The crossover

Figure 7 plots, for each input length, the lowest TTFT limit each side can meet together with the gap limit. Above the blue curve, replicas meet both limits with the largest chunk whose longest step fits the gap limit (the largest admissible chunk); above the orange curve, 1P1D over TCP does. We call the shortest input beyond which the orange curve stays below the blue one the crossover length.

- At 4K the model is close to our measurements: the largest admissible chunk for a 50 ms gap is about 1,600 tokens (TTFT 0.17 s; measured 1,024 tokens, 0.19 s) and for 30 ms about 560 (0.25 s; measured 512, 0.24 s); 1P1D needs 0.33 s at $m = 1.5$ (measured 0.36 s).
- At a 50 ms gap limit the crossover length is about 14K tokens, where both sides need about 0.86 s. Beyond it the admissible chunk shrinks (336 tokens at 32K) and 1P1D over TCP meets TTFT limits that replicas cannot (1.33 against 1.58 s at 20K, 2.5 against 3.5 s at 32K), until about 37K, where the decode node's own longest gap reaches 50 ms.
- At 30 ms the crossover length is about 6K (1.01 against 1.75 s at 16K), and 1P1D's advantage ends near 18K; beyond about 36K no chunk of 128 tokens or more fits.

Point	Quantity	Measured	Model	Role
26B FP8, g7e, 4K, chunk 2,560	longest gap (p90)	62.9	63.4	fit
	TTFT	157	157	sets o
26B FP8, g7e, 4K, chunk 1,024	longest gap (p90)	43.6	41.2	fit
	TTFT	194	189 (−3%)	check
26B FP8, g7e, 4K, chunk 512	longest gap (p90)	27.2	29.0	fit
	TTFT	239	253 (+6%)	check
26B FP8, g7e, 16K, chunk 2,560	longest gap (p90)	150.2	150.3	fit
	TTFT	820	756 (−8%)	check
26B FP8, g7e, 4K, 1P1D, placement group	TTFT	362	349 (−4%)	check
26B FP8, g7e, 4K, 1P1D, no placement group	TTFT	482	500 (+4%)	check
26B FP8, g7e, 16K, 1P1D, no placement group	TTFT	1,286	1,370 (+7%)	check
26B FP8, g7e, 1P1D decode node, 4K / 16K	longest gap (p90)	14.4 / 27.8	–	sets line
26B NVFP4, g7, 4K, chunk 2,560	longest gap (p90)	131.2	131.2	fit
	TTFT	278	278	sets o
26B NVFP4, g7, 16K, chunk 2,560	longest gap (p90)	254.5	254.5	fit
	TTFT	1,430	1,324 (−7%)	check
26B NVFP4, g7, 32K, chunk 2,560	longest gap (p90)	375.7	402.5 (+7%)	extrapolation check
	TTFT	3,507	3,470 (−1%)	extrapolation check
26B NVFP4, g7, 1P1D decode node, 32K	longest gap (p90)	43.3	43.3	extrapolation check

Table 3: Step-time model against measurements, in ms (low load, 0.125–0.5 requests/s per GPU); percentages are model errors. For 1P1D the model uses Eq. (2) with the runs' 2,560-token prefill chunk, $m = 1.5$, and the measured per-flow cap (9.54 Gbps in a placement group, 4.97 Gbps outside); the 28 ms residual and $m = 1.5$ come from session E4's 4K placement-group run, so that check repeats its configuration. The g7 rows solve a two-parameter version (a per-chunk cost proportional to the chunk size, and b) exactly on the 4K and 16K gaps. Script: `harness/boundary_model.py`.

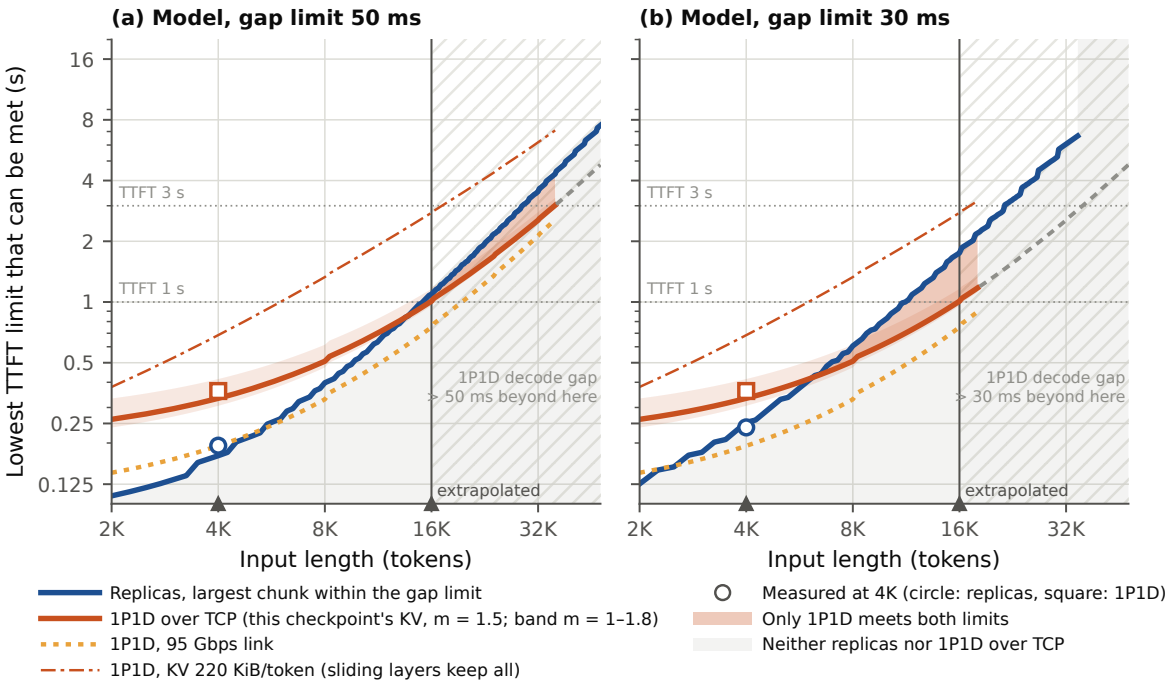


Figure 7: Model prediction for 26B FP8 on g7e at low load: the lowest TTFT limit each side can meet together with the gap limit, by input length. Blue: replicas at the largest chunk whose longest step fits the gap limit. Orange: 1P1D over TCP in a placement group with this checkpoint's KV cache at $m = 1.5$, with a band for $m = 1$–1.8; it turns gray and dashed where the decode node's longest gap exceeds the limit. Dotted: a 95 Gbps link. Thin dash-dot: the transfer volume of a KV cache whose sliding-window layers kept every token. Shaded orange: only 1P1D meets both limits. Gray: neither side meets the TTFT limit. Hollow markers: measured at 4K. Triangles mark the calibration lengths; beyond 16K (hatched) the model extrapolates. The bend in the 1P1D curves at 8K is the prefill node's second step.

- Under our strict SLO (1 s, 50 ms) the model predicts essentially no such region: replicas meet 1 s up to about 15.5K tokens and 1P1D up to about 16K. For a 3 s limit, the longest input is about 30K for replicas and 36K for 1P1D.

6.3 What moves the crossover

- **KV size.** With the transfer volume of a KV cache whose sliding-window layers kept every token (220 KiB per token; 6.9 GiB at 32K against 840 MiB) and step times unchanged, 1P1D meets no TTFT limit that replicas miss at either gap limit before the decode node's limit (dash-dot); with global-layer shapes in every layer (120 KiB per token) the 50 ms crossover moves to about 48K, past the decode node's limit. Global attention would also lengthen the replicas' steps, which we did not model. A small KV cache relative to the link, here from hybrid attention, is what lets disaggregation over TCP take over.
- **Link speed.** With a transfer at 95 Gbps, ten times one TCP flow, the crossover moves to about 5.4K tokens at 50 ms and 2.8K at 30 ms (dotted), at TTFT limits of about 0.2 s; we kept the decode node's measured gap, which a faster link might change.
- **Gap limit.** A tighter limit moves the crossover to shorter inputs (about 6K at 30 ms against 14K at 50 ms), but the decode node's gap ends 1P1D's advantage sooner (18K against 37K).
- **Model error and transfer rate.** The fit has four points for three parameters, and the crossover depends mostly on the per-step cost $d + e$ (16 ms). At 50 ms and $m = 1.5$, refitting without each point in turn moves the crossover from 11K to 23K tokens; a joint fit to all nine gap points we measured for this checkpoint gives 14K (95% range 11K–16K); correcting for the out-of-sample errors at 16K gives 11K; scaling b by 0.9, which removes most of the 32K gap overestimate on g7, gives 15K. With $m = 1$ every value moves later, up to 34K. TTFT differences between chunks imply a per-step cost of 18–19 ms above 1,024 tokens but 11–12 ms below, consistent with CUDA graphs for steps of up to 512 tokens; this leaves the 50 ms crossover unchanged but moves the 30 ms one to about 10K. The model therefore places the crossover between roughly 11K and 34K tokens at 50 ms and 4K and 17K at 30 ms, not at a point.
- **Load.** At equal GPU count the 1P1D prefill node does two GPUs' prefill: at a 50 ms gap limit its prefill capacity per GPU is below the replicas' at every length (0.79 against 0.95 requests/s per GPU at 16K, 0.25 against 0.29 at 32K); at 30 ms it is higher, but the decode node's gap grows with load (at 16K, 48% of requests exceeded 30 ms at 0.25 requests/s per GPU, against 7% at 0.125). Our only 1P1D measurement beyond 16K (g7, 32K, 0.125 requests/s per GPU, before transfer tuning) had a median TTFT of 9.3 s against 3.5 s for replicas, with its prefill node about 85% busy by our estimate. At long inputs, low load means rates below those we measured; we modeled no queueing.
- **Model and GPU.** A lighter model or a faster GPU (smaller a and b) moves the crossover to longer inputs; at 4K, the median longest gap at a 2,560-token chunk was 48 ms for 26B NVFP4 against 62 ms for FP8.

Answer to RQ3. Input length, through the step count. A gap limit caps the chunk, the cap shrinks as the prefix grows, and each extra step costs 11–19 ms. For 26B FP8 on g7e at low load, the model predicts that 1P1D over TCP meets TTFT limits that no admissible chunk meets from about 14K tokens at a 50 ms gap limit (11K–34K within its uncertainty) and about 6K at 30 ms (4K–17K), until the decode node's own longest gap reaches the limit, near 37K and 18K. The

advantage is modest at 50 ms (2.5 against 3.5 s at 32K), needs TTFT limits of about a second or more, and concerns latency at low load, not goodput per GPU. It needs a small KV cache: with the transfer volume of a cache whose sliding-window layers kept every token, it disappears. These are predictions calibrated at 4K and 16K, not measurements.

7 Discussion

7.1 A procedure for small-GPU fleets over TCP

1. **Measure the replicas' longest gap per request, not system percentiles, at the load you plan to serve.** With long inputs it approaches the time of a step carrying one full chunk; with chat it follows the longest co-scheduled prompt. It grows with load: at a 1,024-token chunk the median longest gap grew from 28 ms at 1 request/s per GPU to 42–43 ms at 5.
2. **Use the largest chunk whose longest step fits the gap limit**, not the smallest the stack allows, and check that the stack allows it (Gemma 4 in vLLM v0.30.0 needs every modality disabled). In our 26B runs, 1,024 tokens added about 30 ms of median TTFT and passed 5 requests/s per GPU; 512 tokens added 70–80 ms at 1–2 and reached 359 against 200 ms at 4.
3. **Check the TTFT that chunk gives** with Eq. (4) or a measurement. If it meets the TTFT limit, stop: in our measurements, disaggregation over TCP did not serve more in that case.
4. **Reduce prefill work before moving it.** If requests share prefixes, prefix-aware routing removes prefill without moving any KV cache. If KV caches are reused across nodes, loading a full prompt's cache moves at least $V(L)$ (only the matched prefix on a partial hit) if the store keeps the hybrid layout, which we did not check, and runs at $m \leq 1$ over one connection. For 26B NVFP4 at 4K, NIXL's split pull moved 140 MiB in about 84 ms, against a colocated TTFT of about 130 ms of which prefill is only part; longer prompts favor loading.
5. **Disaggregate over TCP only where no admissible chunk meets both limits** (in our model, 26B FP8 on g7e at a 50 ms gap: inputs beyond roughly 14K tokens, TTFT limits of a second or more, low load) **and the transfer fits**: estimate the TTFT with Eq. (2) using $V(L)$ from Eq. (1), the per-flow cap from one iperf flow, and m measured at the target load (0.57–1.14 in our runs), and compare the offered KV traffic, request rate $\times V(L)$, with the rate transfers reach at that load (our 26B pair queued at 11.7 Gbps offered).
6. **Fix the serving path**: raise UCX's TCP segment size (send and receive together, without pinning socket buffers), keep nodes in a placement group, use proxies that neither cap connections nor pause for garbage collection, and choose the prefill-to-decode ratio for the traffic (at 1:1, 31B chat kept the prefill GPU's streaming multiprocessors active only about 30% of the time).

7.2 RDMA, EFA, and scope

EFA is available on the 8xlarge sizes of the same instance families, without GPUDirect RDMA on g6e and g7e [7], at roughly 1.6–2× the hourly price according to a third-party listing [8]. Those sizes also have four times the vCPUs, and NIXL would use EFA through libfabric rather than UCX over TCP, so the comparison would change more than the network. EFA would shrink the transfer term of Eq. (2), but not the residual, the fixed split between the phases, or the replicas' chunk trade-off; in our model, a transfer ten times faster than one TCP flow moves the crossover to about 3–5.5K tokens. We did not measure EFA. We did not measure EFA. The per-flow cap is an AWS

policy, and NIXL's splitting rule belongs to one version; on other clouds and versions, measure one flow and several flows with iperf first.

8 Limitations

- One model family (Gemma 4) with hybrid attention; models with only global attention have larger KV caches.
- One vLLM version (v0.30.0), one AWS availability zone, at most three nodes, and no RDMA or EFA baseline.
- Section 6 is a model, calibrated on one model and GPU (26B FP8 on g7e) at 4K and 16K inputs and checked against six measurements on g7e and four on g7 that it was not fitted to; beyond 16K it extrapolates, it assumes low load, it omits queueing, and it locates the crossover only to within a factor of about two (roughly 11K–34K tokens at 50 ms). We did not measure tuned chunks on inputs longer than 4K.
- We measured chunks down to 512 tokens for 26B and 256 for 31B.
- Decisive points have one to five runs, and rates are resolved to 1 request/s per GPU; several verdicts are bounded by our highest measured rate.
- The disaggregated side kept most defaults (decode node limited to 256 sequences, NIXL's four threads; the prefill chunk was varied once).
- Sessions before E9 used a replica proxy capped at 100 connections; we excluded the 11 runs that reached the cap (two of them also client-limited), but results from earlier sessions still used 2,560-token replicas.
- No prompt reuse; prefix-cache hits would shorten prefills and transfers. Latency was measured at the client and includes the proxy.
- Tuned-chunk replicas ran with every modality disabled, while the 2,560-token controls and the disaggregated servers kept only images disabled; we did not measure the effect on step time or KV capacity. Some comparisons join sessions from different days, and sessions before E7 used a smaller client instance.
- Long-input requests used random tokens and a fixed 512-token output; real text may route 26B's experts differently.

9 Conclusion

For a fleet of single-GPU instances linked by TCP, tune the chunk first. At the input lengths of most chat and short-document services, a chunk that keeps the longest gap within the limit costs tens of milliseconds of TTFT, and disaggregation over TCP served no more under any SLO we could rank; it is worth considering when the chunk cannot shrink, as with video input on Gemma 4 in vLLM v0.30.0. For long inputs, a gap limit forces chunks so small that the step count dominates TTFT, and our calibrated model predicts that at low load disaggregation over TCP then meets TTFT limits of a second or more that chunked prefill cannot, from roughly 14K tokens at a 50 ms gap limit (11K–34K within the model's uncertainty) until the decode node's own gap reaches the limit, near 37K, provided the KV cache is small, as with hybrid attention. The transfer itself is mostly a matter of software defaults and cloud policy: raise UCX's segment size, keep nodes in

a placement group, and plan on about one flow's rate under load (long transfers reached 1.5–1.8 flows at low load). Measuring the long-input regime directly is the next step.

Use of AI tools

An AI coding assistant (Claude) was used to write measurement and analysis scripts, operate the cloud infrastructure, run four rounds of mock peer review, the first two of which prompted sessions E8 and E9 and the fourth of which reviewed the model of Section 6, draft figures, and draft and translate the text. All experiments ran on real hardware, and the author checked the reported numbers against the raw data.

Funding

This work was carried out at the author's own expense, about $270 of cloud usage.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. arXiv preprint arXiv:2308.16369, 2023.

[2] Amey Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

[3] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[4] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. doi: 10.1109/ISCA59077.2024.00019.

[5] Ruoyu Qin, Zheming Li, Weiran He, et al. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.

[6] Amazon Web Services. Specifications for Amazon EC2 accelerated computing instances. `https://docs.aws.amazon.com/ec2/latest/instancetypes/ac.html`, 2026. Accessed 2026-09-30.

[7] Amazon Web Services. Elastic fabric adapter for AI/ML and HPC workloads on Amazon EC2 (Amazon EC2 user guide). `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html`, 2026. Accessed 2026-09-30.

[8] Vantage. EC2 instance price listing: g7e.8xlarge and g6e.8xlarge (us-east-1, on demand). `https://instances.vantage.sh/`, 2026. Third-party listing, accessed 2026-09-30; not checked against the AWS price list.

[9] llm-d project. Disaggregated serving (documentation, v0.9). https://llm-d.ai/docs/architecture/advanced/disaggregation, 2026.

[10] NVIDIA. Disaggregated inference communication guide (Dynamo v1.3.0 documentation). https://docs.nvidia.com/dynamo/v1.3.0/kubernetes-deployment/operate/disagg-communication, 2026.

[11] llm-d project. llm-d architecture: Prefix-cache aware routing. https://llm-d.ai/docs/architecture, 2026.

[12] Yuhan Liu, Yihua Cheng, Jiayi Yao, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. arXiv preprint arXiv:2510.09665, 2025.

[13] Cunchen Hu, Heyang Huang, Liangliang Xu, et al. Inference without interference: Disaggregate LLM inference for mixed downstream workloads. arXiv preprint arXiv:2401.11181, 2024.

[14] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.

[15] Yibo Jin, Tao Wang, Huimin Lin, et al. P/D-Serve: Serving disaggregated large language model at scale. arXiv preprint arXiv:2408.08147, 2024.

[16] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. KVDirect: Distributed disaggregated LLM inference. arXiv preprint arXiv:2501.14743, 2025.

[17] Ke Hong et al. semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage. arXiv preprint arXiv:2504.19867, 2025.

[18] Chao Wang, Pengfei Zuo, Zhangyu Chen, et al. Prefill-decode aggregation or disaggregation? Unifying both for goodput-optimized LLM serving. arXiv preprint arXiv:2508.01989, 2025.

[19] vLLM project. Disaggregated prefilling (experimental). https://docs.vllm.ai/en/latest/features/disagg_prefill/, 2026.

[20] Jiangsu Du, Hongbin Zhang, Taosheng Wei, et al. Efficient LLM serving on commodity GPU clusters with data-reduced cross-instance orchestration. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, 2026. Preprint: EcoServe, arXiv:2504.18154.

[21] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, et al. ThunderServe: High-performance and cost-efficient LLM serving in cloud environments. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2502.09334.

[22] Youhe Jiang, Ran Yan, and Binhang Yuan. HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.07903.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[24] NVIDIA. NIXL: NVIDIA inference xfer library. `https://github.com/ai-dynamo/nixl`, 2026.

[25] Pavel Shamis, Manjunath Gorentla Venkata, M. Graham Lopez, et al. UCX: An open source framework for HPC network APIs and beyond. In *IEEE 23rd Annual Symposium on High-Performance Interconnects (HOTI)*, pages 40–43, 2015. doi: 10.1109/HOTI.2015.13.

[26] Yuhan Liu, Hanchen Li, Yihua Cheng, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. doi: 10.1145/3651890.3672274.

[27] Ruoyu Qin, Weiran He, Yaoyu Wang, Zheming Li, et al. Prefill-as-a-Service: KVCache of next-generation models could go cross-datacenter. arXiv preprint arXiv:2604.15039, 2026.

[28] Yida Wang, Xiuhong Li, Jianping Ma, Gan Sun, Yunshen Xu, et al. PDD: Unleashing economical and flexible heterogeneous LLM inference via cross-datacenter prefill-decode disaggregation. arXiv preprint arXiv:2609.13161, 2026.

[29] Pravein Govindan Kannan, Liran Schour, Aleksander Slominski, Raj Joshi, Nicolò Lucchesi, Carlos Costa, Moein Khazraee, and Omri Kahalon. Networking for distributed inference in llm-d. llm-d blog, `https://llm-d.ai/blog/networking-for-distributed-inference-llm-d`, 2026. Published June 23.

[30] Tiyasa Mitra et al. Beyond the Buzz: A pragmatic take on inference disaggregation. arXiv preprint arXiv:2506.05508, 2025.

[31] Amey Agrawal, Anmol Agarwal, Nitin Kedia, et al. Etalon: Holistic performance evaluation framework for LLM inference systems. arXiv preprint arXiv:2407.07000, 2024.

[32] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, et al. Andes: Defining and enhancing quality-of-experience in LLM-based text streaming services. arXiv preprint arXiv:2404.16283, 2024.

[33] Amazon Web Services. Amazon EC2 instance network bandwidth (Amazon EC2 user guide). `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html`, 2026.

[34] Google. Gemma 4 model card. `https://ai.google.dev/gemma/docs/core/model_card_4`, 2026.

[35] NVIDIA. Gemma-4-26B-A4B-NVFP4 and Gemma-4-31B-IT-NVFP4 model checkpoints. `https://huggingface.co/nvidia/Gemma-4-26B-A4B-NVFP4`, `https://huggingface.co/nvidia/Gemma-4-31B-IT-NVFP4`, 2026.

[36] ShareGPT_Vicuna_unfiltered: ShareGPT_V3_unfiltered_cleaned_split.json. `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered`, 2023.

[37] Microsoft Azure. Azure LLM inference trace 2023 (AzurePublicDataset). `https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md`, 2023.

[38] Jakob Nielsen. *Usability Engineering*. Academic Press, 1993. Chapter 5; summarized in "Response Times: The 3 Important Limits", Nielsen Norman Group.

[39] vLLM project. Issue #52607: vLLM's default NIXL connector is ~100× slower than the push connector on non-RDMA hardware. `https://github.com/vllm-project/vllm/issues/52607`, 2026.

A Results from earlier sessions

These results come from sessions before E8 and used 2,560-token replicas and the capped replica proxy; we report them with those caveats. Rates printed inside the figures are totals for two GPUs unless the axis says per GPU; the captions give per-GPU rates.

- **Three nodes** (Figures A4 and A5). Neither 1P2D nor 2P1D exceeded 1P1D per GPU for 26B with 4K inputs (1P2D: 0.906 at 3 requests/s per GPU; 2P1D: 0.77), and three 2,560-token replicas passed 1 request/s per GPU. We draw no rule for the prefill-to-decode ratio. For 31B with 4K inputs no configuration held a 1 s TTFT reliably: a 4K prefill takes about 480 ms and the transfer about 280 ms.
- **Input length** (g7, before transfer tuning; Figure A2). At 0.25 requests/s per GPU with 4K inputs, 1P1D reached 0.96 and replicas 0.23. At 16K, prefill alone pushed the replicas' TTFT over 1 s and the transfer pushed 1P1D's over, so neither met the strict SLO; under the relaxed SLO 1P1D reached 0.47 and replicas 0.17. At 32K all failed.
- **Weight precision** (g7e, same day; Figure A3). At 0.5 requests/s per GPU with 4K inputs, 2,560-token replicas reached 0.97 (NVFP4), 0.13 (FP8), and 0.03 (BF16), and 1P1D 1.00 in all three. Section 4.2 shows that a 512- or 1,024-token chunk rescues FP8.
- **Real trace** (Figure A6). Replaying the Azure conversation trace, all 26B configurations passed (4–5.9 requests/s in total); 1P1D's TTFT was about 120 ms higher. For 31B at 3.7 requests/s in total, both failed: 2,560-token replicas on gaps (0.05) and 1P1D on the TTFT tail (0.28).
- **Other GPUs** (Figure A9). Against 2,560-token replicas at 0.5 requests/s per GPU, 1P1D reached 0.83–1.00 and the replicas 0.02–0.97 across GPUs; only g7e passed for either, and we did not test smaller chunks on g6e or g7.
- **Repeats.** Points measured three or more times varied as described in Section 3.6 (Figure A7).
- **Correctness.** With greedy decoding and Gemma's chat template, disaggregated 31B gave the same answers as a single node; long outputs diverged only in late, near-tied tokens (30–160 characters in), and push matched pull byte for byte on 8 of 8 prompts after a restart. 26B was not deterministic even on a single node, so we used a gate (short outputs byte-identical; long outputs sharing at least 40% of words of five or more letters, calibrated at $\geq$0.58 for true pairs and $\leq$0.17 for swapped prompts); all disaggregated outputs passed.

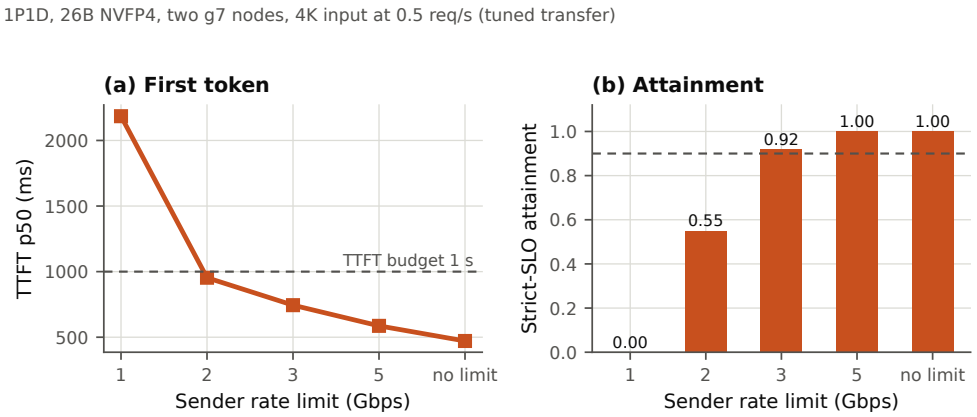


Figure A1: 1P1D on two g7 nodes, 26B NVFP4, 4K input at 0.25 requests/s per GPU, with the sender's aggregate bandwidth limited by a token-bucket filter. (a) Median TTFT. (b) Strict-SLO attainment. After transfer tuning.

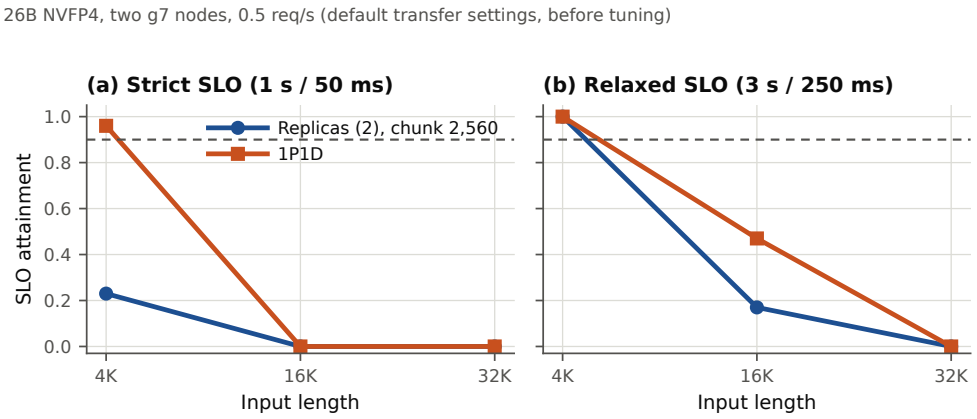


Figure A2: Attainment by input length on two g7 nodes, 26B NVFP4, 0.25 requests/s per GPU, before transfer tuning. (a) Strict SLO. (b) Relaxed SLO.

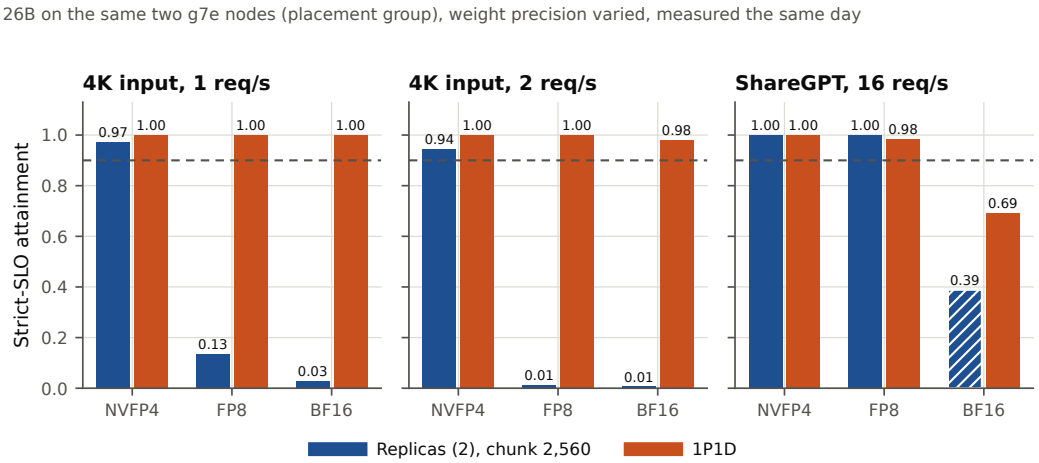


Figure A3: Weight precision varied on the same two g7e nodes in a placement group, 26B, same day, 2,560-token replicas. Panel titles give total rates (0.5 and 1 request/s per GPU for 4K input, 8 for ShareGPT). Hatched: caution run (the replicas' proxy reached its connection cap).

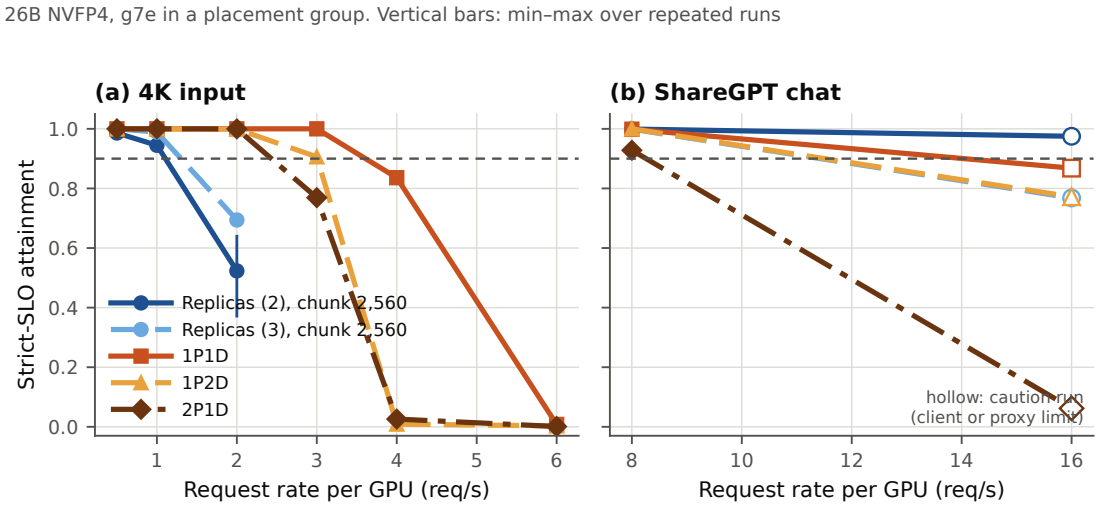


Figure A4: 26B NVFP4 on g7e in a placement group with two and three nodes (sessions E5 and E7, test proxy, 2,560-token replicas). (a) 4K input. (b) ShareGPT chat; hollow markers are caution runs (client or proxy limit).

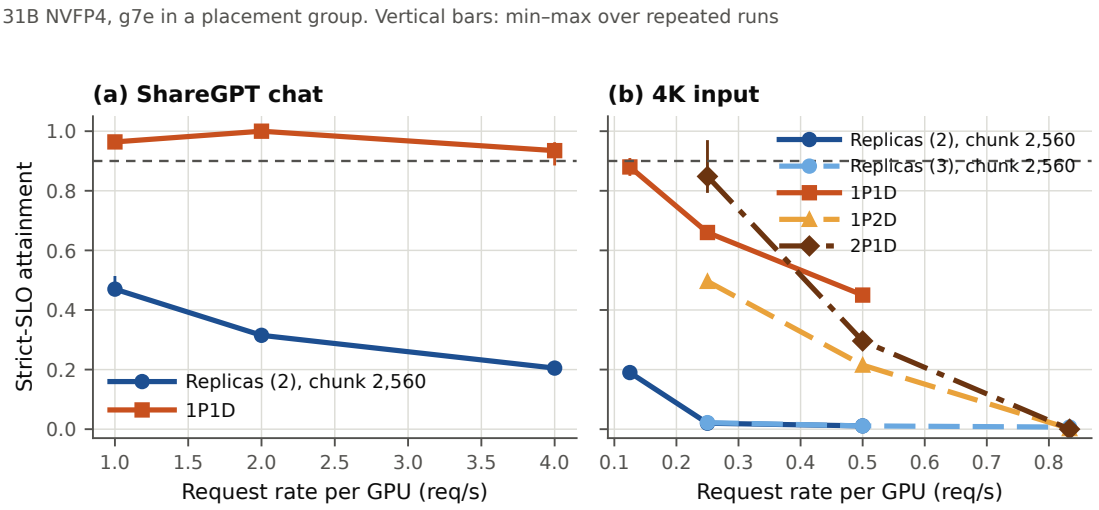


Figure A5: 31B NVFP4 on g7e in a placement group with two and three nodes (sessions E5 and E7). (a) ShareGPT chat. (b) 4K input.

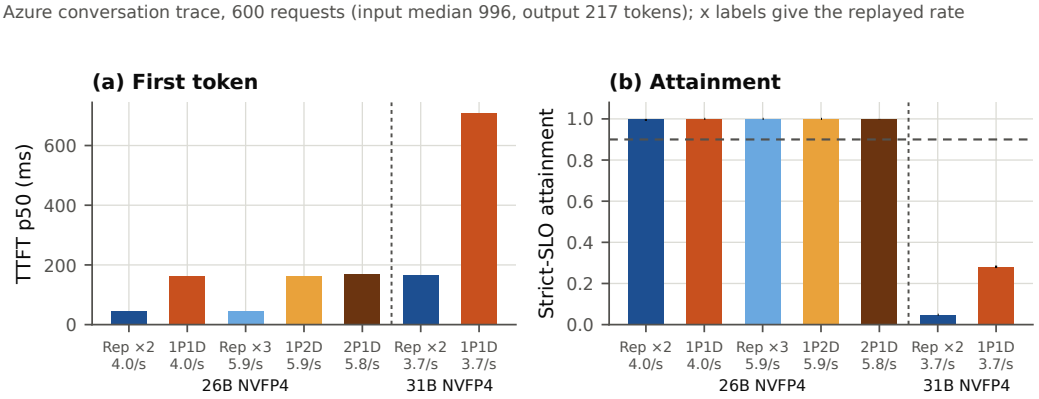


Figure A6: Azure conversation trace, 600 requests. (a) Median TTFT. (b) Strict-SLO attainment. Numbers under the axis give the replayed total rate; left of the dotted line is 26B, right is 31B.

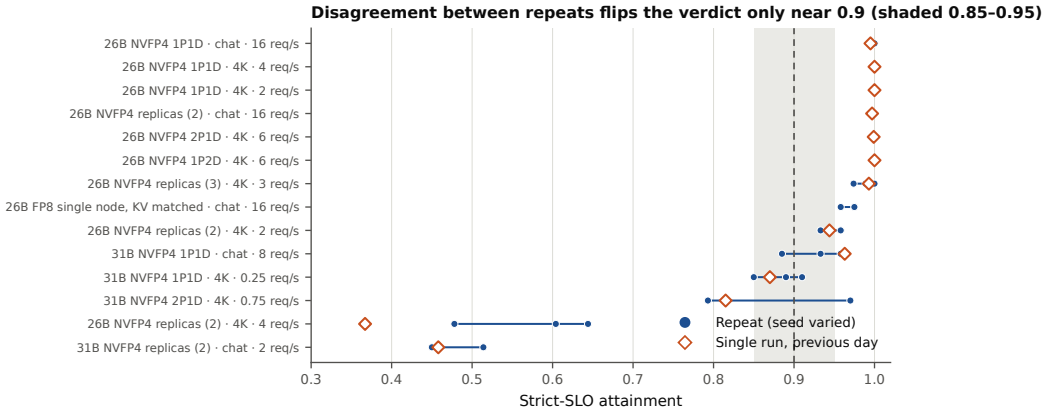


Figure A7: Attainment at points measured at least three times with different seeds (session E7) and the previous day's run (E5). Row labels give total rates. The gray band is 0.85–0.95.

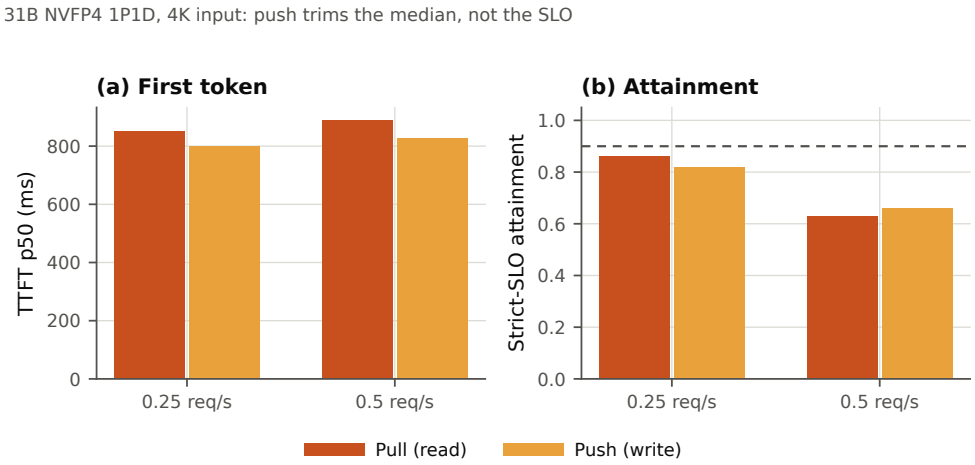


Figure A8: 31B NVFP4 1P1D, 4K input, pull versus push transfer at 0.125 and 0.25 requests/s per GPU. (a) Median TTFT. (b) Strict-SLO attainment.

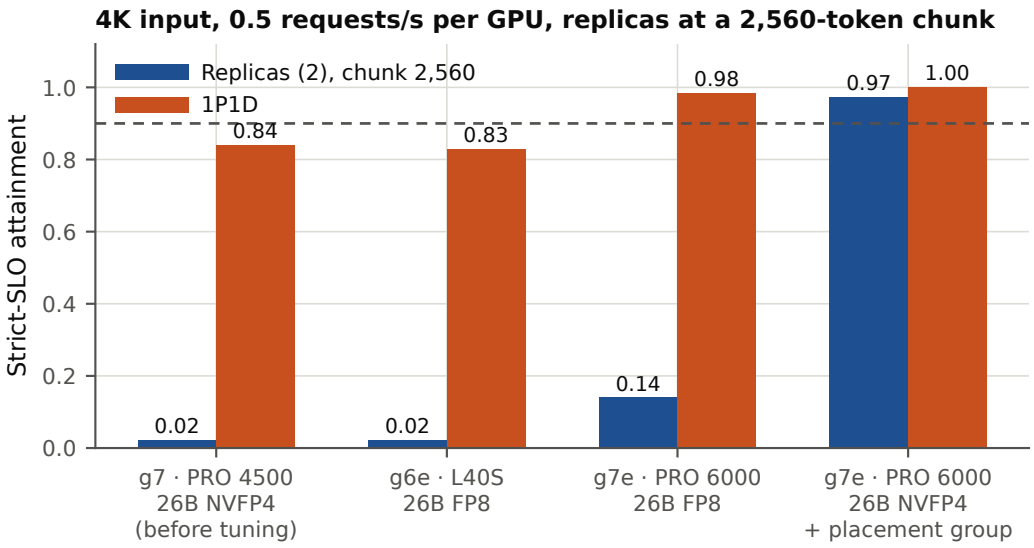


Figure A9: Strict-SLO attainment of two-node configurations at 4K input and 0.5 requests/s per GPU by GPU and precision, with 2,560-token replicas. The g7 measurement predates transfer tuning.

B Practical pitfalls

B.1 Proxies that distort the measurement

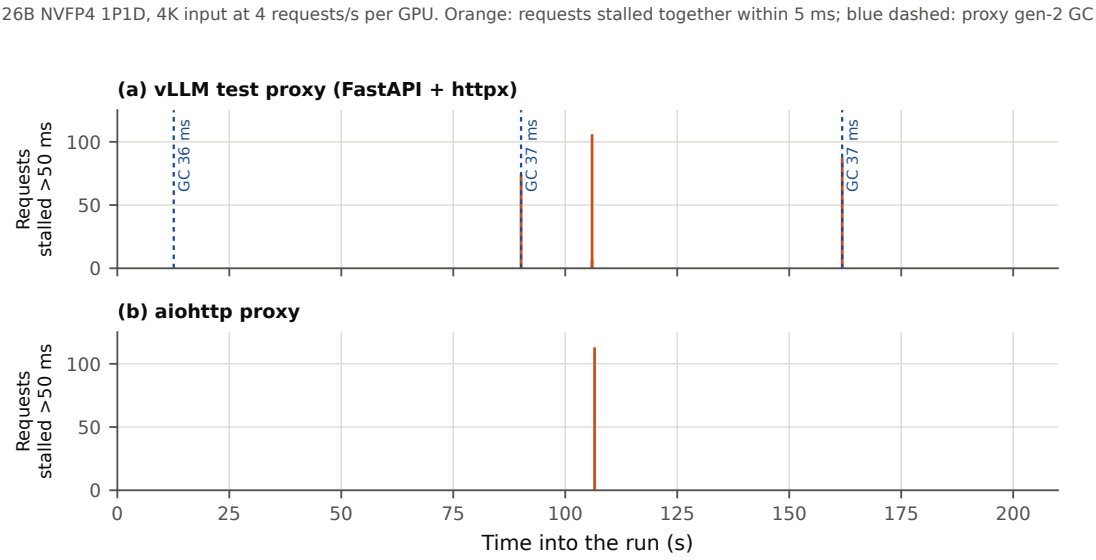


Figure A10: Global stalls in two 1P1D runs with the same seed (session E8; 26B, 4K input, 4 requests/s per GPU). Orange: number of requests whose output stream paused for more than 50 ms within the same 5 ms. Blue dashed: second-generation garbage collections in the proxy. (a) vLLM's test proxy. (b) Our aiohttp proxy.

Garbage collection in the disaggregated proxy. Near their limits, disaggregated runs showed global pauses, which we saw in none of the five uncapped 1,024-token replica runs at 4–5 requests/s

per GPU: up to five times per run at 4 requests/s per GPU, 16–113 in-flight requests (typically 35–110) paused together, for a median of 50–80 ms. In E8 we instrumented both proxies to log upstream stream gaps, event-loop lag, and garbage collection, and snapshotted the decode server's inter-token latency histogram before and after each point. For 26B 1P1D with vLLM's test proxy, four of five such events in three runs coincided within 100 ms with a second-generation garbage collection of 25–40 ms that blocked the proxy's event loop, after which the paused streams received a burst of tokens (Figure A10a). With our aiohttp proxy, second-generation collections took 9 ms and the loop never lagged. One event remained in three runs; the proxy received it from upstream, the decode server logged it as 113 gaps over 50 ms, and it appears at the same time in the test-proxy run with the same seed, so it is most likely a server-side pause. For 31B chat, the events with our aiohttp proxy were also server-side: the decode server logged 110 and 316 gaps over 50 ms, and the proxy showed no collection or lag. We instrumented only session E8, so we do not attribute earlier disaggregated failures, such as the three-node 2P1D collapse, to the proxy.

A connection limit in the replicas' proxy. The replicas' round-robin proxy used an HTTP client whose default limit is 100 concurrent connections. At 4–6 requests/s per GPU in E8, every TTFT failure of 1,024-token replicas came from a request that arrived while 100 or more requests were in flight (33 of 33 at 4, 393 of 393 at 5), and no request failed on gaps at 5. With the limit removed in E9, the same configuration reached 1.000 at 4 and 0.999–1.000 at 5. The cap had lowered the replicas' supported load by one step and inflated their TTFT tail. It also shortened the gaps, because fewer requests decoded at once: at 5 requests/s per GPU the median inter-token latency was 17.3 ms with the cap and 30.3–31.8 ms without it. We therefore exclude capped runs even where they passed.

B.2 Other pitfalls

- Gemma 4 in vLLM v0.30.0 rejected chunks below 2,496 tokens with images, or images and audio, disabled, and accepted them with images, audio, and video all disabled; with all three disabled, a 2,560-token chunk fails at engine start.
- aiohttp's client limits a session to 100 concurrent connections by default; a proxy built on it queues requests silently beyond that.
- vLLM's test proxy for NIXL pauses all streams during garbage collection (Appendix B.1); our aiohttp proxy had one keep-alive race in 1,440 requests.
- If UCX's TCP receive segment is smaller than the send segment, the engine refuses to start; raise both. In our runs, setting `UCX_TCP_SNDBUF` and `UCX_TCP_RCVBUF` to 8 MB left the receiver with a TCP window scale of 2, which caps the window at 256 KiB; check the kernel's `net.core.rmem_max` and `wmem_max` first, or leave them unset.
- At high rates the proxy hit the operating system's file-handle limit (1,024 by default); a disaggregated proxy opens two connections per request and hits it sooner.
- Trace replay in vLLM v0.30.0 without an explicit request count builds an empty request list.
- Replaying the same seeded requests against a server with prefix caching enabled makes later points hit the cache.
- New cloud workspaces create separate security groups; the client needs an explicit ingress rule to reach new nodes. Cluster placement groups may take 15–45 minutes to find capacity for a second node.

C Transfer settings

The tuned set used in sessions E4–E9, except the three E9 runs with an 8,192-token prefill chunk, which set only the two segment sizes:

- `UCX_TCP_TX_SEG_SIZE=256k`, `UCX_TCP_RX_SEG_SIZE=256k`
- `UCX_TCP_MAX_IOV=16`, `UCX_TCP_MAX_POLL=-1`
- `UCX_TCP_SNDBUF=8m`, `UCX_TCP_RCVBUF=8m`
- `UCX_RNDV_FRAG_SIZE=host:4M,cuda:8M`
- with `UCX_TLS=tcp,cuda_copy,cuda_ipc` and `UCX_NET_DEVICES=all` in every disaggregated run

Session E8 showed that the two segment sizes alone give a slightly higher rate at low load (Section 4.3). The set requested 8 MB socket buffers; we did not change or record the kernel's socket limits, and the receiver's TCP window scale was 2 in both seven-variable runs we sampled, against 7 with the segment sizes alone or the defaults.

NIXL's split. NIXL 1.4.1 splits a transfer's descriptor list into pieces of $\max(N/4, 1{,}024)$ descriptors, the last one smaller, so a list of fewer than 1,024 descriptors goes out as one piece. A 26B 4K transfer has about 1,900–2,240 descriptors at low load and goes out in two or three pieces, a 31B 4K transfer (about 4,000–4,500) in four, and transfers under 1,024 descriptors (26B inputs under about 590 tokens, 31B under about 290) as a single piece on a shared connection.

Other settings did not help. Staging the KV cache through host memory slowed TTFT 4.7× at 0.125 requests/s per GPU on g7 and had no stable load at 0.5. Push transfer (Figure A8), in which the prefill node writes the KV cache, cut 31B's transfer time by 27–36% and median TTFT by 53–62 ms but changed attainment by at most 0.04; its logged volume was 468–533 MiB per request against 560 for pull, which we could not explain. Within one node over PCIe, push has been reported about 100× faster than pull [39]; we saw no such gap across nodes. A proposed vLLM patch that overlaps the transfer with prefill layer by layer[1] supports only models with a single KV-cache group; with hybrid-attention Gemma 4 it stalled on the first request. Network credits were not a constraint: g7e.2xlarge has a fixed 50 Gbps, and on g6e.2xlarge (5 Gbps baseline, 20 Gbps burst) our 15-minute test load averaged about 1.2 Gbps, computed from the request rate and volume, below the baseline.

D Deviations from the pre-registered protocol[2]

- Gap limit: five times each model's decode step became a common 50 ms (Section 3.5). One-token outputs pass the gap criterion since session E8.
- Chunk size: 2,048 was planned; Gemma 4 forced 2,560 until sessions E8 and E9 swept 256–2,048.
- Proxy: one proxy for both approaches was planned; replicas and disaggregated runs used different proxies, and the replicas' proxy was capped at 100 connections until E9.
- Equivalence band: 0.9–1.1 was planned; Figure 6 uses 0.8–1.25.
- Confidence intervals: a request-level bootstrap was planned; we report hierarchical bootstrap intervals for decisive points (Appendix F).

[1] `https://github.com/vllm-project/vllm/pull/56874`

[2] Protocol v1.7, committed to our local repository on 24 September 2026 before the first session; not externally timestamped.

- Client: a fixed c6i.2xlarge was planned; sessions before E7 used c6i.xlarge. Warm-up was implemented from E7 on.
- Reference points on H100 and on a two-GPU g5 node were not measured.
- The step-time model of Section 6 is a post-hoc analysis that the protocol did not include.
- UCX: the three E9 runs with an 8,192-token prefill chunk set only the two segment sizes, which the E9 predictions stated.

E Session E9 predictions

Session E9's predictions (five items, Table E1) were committed to our then-private repository at 13:29 UTC on 30 September 2026 (commit `8775497` in our private repository; [hash in the public repository]), together with the run scripts, which did not change afterwards; the session started 46 minutes later. The timestamps are recorded by git on our machine and are not externally anchored. Three predictions held as far as we tested them. In the fourth, both verdicts held (31B replicas passed 8 requests/s per GPU and 1P1D failed at 6), but the seed-10000 1P1D run at 4 reached 1.000 rather than the predicted 0.85–0.95. The fifth (1P1D with an 8,192-token prefill chunk passing 4) did not hold (0.874 and 0.976). The two misses went in opposite directions. The prediction that 512-token replicas would not exceed 1,024-token ones was not tested above 4 requests/s per GPU. Most predictions extrapolated the second mock review's analysis and carried little risk; their value lies less in the hit rate than in fixing the run list and the verdict rule in advance and reporting all 19 runs.

	Prediction	Measured	Outcome
1	26B, 512-token replicas: median longest gap 15–25 ms at 1–2 requests/s per GPU; median TTFT 40–80 ms above the 1,024-token chunk; pass 30 ms at 2 or more; supported load 3–4 at 50 ms, not above 1,024 tokens	21.4–23.9 ms; +44 to +51 ms; 1.000 at 2 and 3; passed 4 (0.985), 5 not measured	held as tested
2	26B, 1,024-token replicas, uncapped: pass 4 in all three repeats and 5; fail 6	1.000 ×3; 0.999 and 1.000; 0.188	held
3	26B FP8, 512-token replicas: median longest gap 25–32 ms; pass 40 ms at 0.5 and 1; at 30 ms pass 0.5, uncertain at 1	26.8–28.0 ms; passed; 1.000 at 0.5, 0.958 at 1	held
4	31B chat: 256-token replicas pass 8; 1P1D (aiohttp) fails 6; its seed-10000 run at 4 borderline (0.85–0.95)	1.000; 0.765; 1.000	verdicts held, range missed
5	26B 1P1D (aiohttp), prefill chunk 8,192, UCX segment sizes only: TTFT falls at 4 and 5; 5 still fails; 4 passes	416–451 against 477–526 ms at 4, 940 against 1,447 ms at 5; 0.365; 0.874 and 0.976	missed

Table E1: Session E9 predictions, as committed before the session (translated and condensed), and the measurements. Rates are per GPU; verdicts use the strict SLO.

F Decisive points

Point (sessions)	per GPU	gap limit	runs	attainment (each run), 95% interval
26B NVFP4 4K, replicas chunk 2,560 (E5, E7, E8)	1	50 ms	5	0.944, 0.933, 0.958, 0.944, 0.950; 0.922–0.967
26B NVFP4 4K, replicas chunk 2,560 (E5, E7, E8)	2	50 ms	5	0.369, 0.644, 0.607, 0.478, 0.597; 0.429–0.636
26B NVFP4 4K, 1P1D aiohttp (E8)	3	50 ms	1	1.000; 1.000–1.000
26B NVFP4 4K, replicas chunk 1,024 (E9)	4	50 ms	3	1.000, 1.000, 1.000; 1.000–1.000
26B NVFP4 4K, replicas chunk 1,024 (E9)	5	50 ms	2	0.999, 1.000; 0.998–1.000
26B NVFP4 4K, replicas chunk 512 (E9)	4	50 ms	1	0.985; 0.965–0.999
26B NVFP4 4K, 1P1D aiohttp (E8)	4	50 ms	3	0.954, 0.992, 0.885; 0.873–0.992
26B NVFP4 4K, 1P1D test proxy (E7, E8)	4	50 ms	4	0.836, 0.900, 0.975, 0.767; 0.774–0.953
26B NVFP4 4K, 1P1D aiohttp, prefill chunk 8,192, UCX segment sizes only (E9)	4	50 ms	2	0.874, 0.976; 0.827–0.988
26B NVFP4 4K, replicas chunk 512 (E9)	2	30 ms	2	1.000, 1.000; 1.000–1.000
26B NVFP4 4K, replicas chunk 512 (E9)	3	30 ms	1	1.000; 1.000–1.000
26B NVFP4 4K, 1P1D test proxy (E5, E7)	2	30 ms	4	0.954, 1.000, 0.919, 0.958; 0.912–0.993
26B NVFP4 4K, 1P1D test proxy (E7)	3	30 ms	1	0.758; 0.602–0.892
26B NVFP4 4K, 1P1D aiohttp (E8)	3	30 ms	1	0.771; 0.621–0.901
26B FP8 4K, replicas chunk 512 (E9)	1	30 ms	1	0.958; 0.900–1.000
26B FP8 4K, 1P1D test proxy (E7)	1	30 ms	1	1.000; 1.000–1.000
31B chat, replicas chunk 256 (E9)	8	50 ms	1	1.000; 1.000–1.000
31B chat, 1P1D aiohttp (E8, E9)	4	50 ms	3	0.924, 0.903, 1.000; 0.887–1.000
31B chat, 1P1D test proxy (E5, E7)	4	50 ms	4	0.963, 0.933, 0.958, 0.885; 0.888–0.970
31B chat, 1P1D aiohttp (E9)	6	50 ms	1	0.765; 0.666–0.855

Table F1: Attainment with TTFT ≤1 s and the listed gap limit at the points that decide the comparisons, with 95% intervals from a hierarchical bootstrap (resample runs, then 20 time blocks within each run). Runs are listed in repeat order. Intervals are for the attainment pooled across repeats; the verdict uses the minimum over repeats. Rows with one run have within-run intervals only, and 1.000–1.000 means no failure was observed, not certainty. With two to five repeats the intervals are descriptive. The 26B test-proxy row at 4 requests/s per GPU contains two runs with seed 8007 (E7 and E8). Script: `harness/decisive_ci.py`.

G Data and code

We release the measurement harness, configurations, analysis scripts, the four rounds of mock review with their scripts, and per-request raw data [GitHub URL and Zenodo DOI]. [SHA-256 of the bundle of the original private repository.] Raw data are stored per session (`results/<session>/`).

Session	Date (UTC)	Instances	Models	Main content
B	2026-09-25	2×g7	26B NVFP4	input-length grid
D	2026-09-26	2×g7	26B NVFP4	transfer tuning, bandwidth limit
E1	2026-09-26	g6e, g7e	26B FP8, 31B FP8	cross-GPU, KV matching
E2	2026-09-26	2×g7e	26B FP8	two-node comparison, 16K
E3	2026-09-27	2×g6e	26B FP8	L40S pair, network credits
E4	2026-09-27	2×g7e, PG	26B FP8	placement group, transfer settings
E5	2026-09-27	2–3×g7e, PG	26B, 31B NVFP4	scaling, transfer estimates
E6	2026-09-27	2×g7e, PG	26B, 31B NVFP4	push transfer, correctness
E7	2026-09-28	2–3×g7e, PG	26B (3 precisions), 31B	repeats, trace, precision
E8	2026-09-29/30	2×g7e, PG	26B NVFP4/FP8, 31B	chunk sweep, proxy instrumentation, socket counts, UCX ablation
E9	2026-09-30	2×g7e, PG	26B NVFP4/FP8, 31B	512-token chunk, uncapped replica proxy, prefill chunk 8,192, 31B limits

Table G1: Measurement sessions (PG: cluster placement group). Sessions A, A′, C1, and P are prefix-cache-contaminated or setup runs and are not used.